STAC AI BOOTCAMP NYC | 21 OCTOBER 2026



Building Trust in Agentic AI: From Evidence to Action

Matthew Dixon, PhD, FRM
BeliefLens

matthew.dixon@belieflens.org

Bootcamp agenda · New York time

Time	Session
9:30–10:15	Financial language-model architectures
10:15–11:15	Building grounded financial RAG
11:15–11:30	Break
11:30–12:30	Post-training for financial applications
12:30–1:15	Lunch
1:15–2:15	Hands-on adaptation lab
2:15–3:00	LLMs and SLMs for alpha and risk
3:00–3:15	Break
3:15–4:15	Evaluation and evidence attribution
4:15–5:00	Deployment and governance clinic

Is AI fooled by randomness?



$$
\begin{array}{cccccccccc}
+1 & -1 & +1 & -1 & -1 & -1 & +1 & -1 & +1 & -1 \\
-1 & -1 & +1 & +1 & +1 & +1 & +1 & -1 & +1 & +1 \\
+1 & -1 & -1 & +1 & -1 & -1 & +1 & +1 & -1 & +1 \\
-1 & +1 & +1 & +1 & -1 & -1 & -1 & -1 & +1 & -1 \\
-1 & -1 & -1 & +1 & -1 & +1 & +1 & +1 & -1 & +1
\end{array}
$$

$+1$: green $\quad -1$: red

Question: What, if anything, can you infer about the next value?

Which sequences show significant autocorrelation?

TWO-SIDED PERMUTATION TEST · 4,000 SHUFFLES · MODEL DOES NOT SEE THESE RESULTS

Sequence	Lag-1 r	Lag-1 p	Max lags 1–5 p	At 5%	At 1%
1: random draws	-0.062	0.720	0.829	No rejection	No rejection
2: random draws	$+0.387$	0.013	0.045	Reject	No rejection
3: random draws	-0.185	0.208	0.502	No rejection	No rejection
4: alternating	-1.000	0.00025	0.00025	Reject	Reject

Sequence 4: $-1, +1, -1, +1, \ldots$; 50 values, ending $+1$. Next under this rule: -1.

r: sample correlation between neighbouring values (lag 1). p: permutation-test tail probability under the exchangeable-order null.

5% significance: reject when $p \leq 0.05$. 1% significance: reject when $p \leq 0.01$.

No rejection means no significant dependence detected by this test. It does not prove randomness or independence.

A blind prompt; a separate statistical check

Exact question after the 50 numbers

Predict the next number in the sequence. Respond only with +1 or -1.

Forced sign only. No generator disclosed. No test results shown.

Compute lag-1 Pearson correlation on adjacent values. Shuffle the whole sequence 4,000 times and check how often the shuffled correlation is at least as strong as the observed correlation.

For lags 1–5, use the maximum absolute correlation in each shuffle.

Why not the ordinary Pearson p-value? Binary observations and overlapping pairs do not match its standard independent-normal-pairs assumptions.

Null: all arrangements of the observed values are equally plausible.

First, let the model explain its answer

Original open-ended prompt after the 50 numbers

What, if anything, can you infer about the next value in this sequence? Explain your reasoning.

Input	Qwen3.5-9B	Llama-3.3-70B	GPT-OSS-120B
Random sequence 1	Indeterminate; 50/50 if fair	Tentative $+1$; states 52%	Indeterminate; 50/50 if fair
Random sequence 2	Roughly 50/50; no consistent rule	Tentative $+1$; run might continue	No reliable prediction
Random sequence 3	No certainty; states 50/50	Tentative -1; supposed grouping	Indeterminate; a specific sign is a guess

Next, require a single answer: $+1$ or -1. Which does the model choose?

Forced prediction: what did the models choose?

Exact instruction after the same 50 numbers

Predict the next number in the sequence. Respond only with +1 or -1.

Input	Qwen3.5-9B	Llama-3.3-70B	GPT-OSS-120B
Random sequence 1	+1	+1	−1 / +1
Random sequence 2	+1	+1	−1
Random sequence 3	+1	−1	−1
Alternating control	−1	−1	−1

Two identical-prompt repeats. Only GPT-OSS sequence 1 changed sign.

A forced guess does not establish that a random sequence is predictable.

A strong answer preference is not a reliable forecast

SAME FORCED-PREDICTION PROMPT · LLAMA-3.3-70B

Input	Generated sign	Score for +1	Score for −1
Random sequence 1	+1	99.13%	0.87%
Random sequence 2	+1	85.20%	14.80%
Random sequence 3	−1	6.01%	93.99%
Alternating control	−1	18.24%	81.76%

These scores compare the likelihood of writing the two signs.

For the independent fair generator, the next random draw remains 50/50.

Sequence 1's +1 score was 97.70% on repeat; the sign stayed unchanged.

From a plausible answer to a trustworthy action

If an AI can misread 50 numbers, what must we verify before it acts on a financial decision?

- **Evidence:** Did it use the right facts, dates and sources?
- **Reliability:** Does the answer survive independent checks and controlled tests?
- **Authority:** Is the proposed action permitted, with an accountable owner?

Python notebooks: the default for every exercise

Open the supplied `STAC_AI_Bootcamp` folder in Terminal. Keep the notebooks, helper files and data together.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m jupyterlab notebooks
```

Start with `01_architectures.ipynb`. Read the guidance, then run cells from top to bottom with **Shift + Enter**.

The supplied exercises use local or recorded examples by default. Fresh model calls and GPU training are separate, optional activities.

Windows: activate with `.venv\Scripts\activate`.

One financial language-model workflow across the day

- Choose a model and give it the evidence it is allowed to use.
- Compare base, retrieval-augmented and adapted configurations.
- Test what changes, what remains unsupported and who may act.
- Choose one workflow to trial, with agreed evidence checks, permitted actions and a named reviewer.

STAC AI BOOTCAMP NYC



Financial language-model architectures

Matthew Dixon, PhD, FRM
BeliefLens

21 October 2026 | 9:30–10:15

matthew.dixon@belieflens.org

What Section 1 adds

Which component does what in a financial language-model workflow?

- **Covered in the opening:** a plausible response still needs checking.
- **Section 1:** choose the model, context, tools and control flow.
- **Section 2:** find, extract and supply the required information.

Why go deeper in Section 2? The right architecture can still use the wrong source or number.

Python notebooks: `01_architectures.ipynb`; companion `01b_harbor_tools_mcp_permissions.ipynb`.

Five things you will be able to explain

- What the model receives and what it returns.
- How model weights differ from runtime context.
- How fixed RAG and agentic RAG organise the same capabilities.
- Where retrieval, a calculator and application controls fit.
- When to retrieve current information or consider further training.

Start with the language-model boundary — 1 of 3

1. The model receives an input	Instructions, the question and any supplied evidence are assembled into what the model can read.

Why this matters: A prompt gives instructions; application integration makes tool execution possible.

Start with the language-model boundary — 2 of 3

1. The model receives an input	Instructions, the question and any supplied evidence are assembled into what the model can read.
2. The model can request a tool	A request names a tool and its arguments. The model has not executed the function merely by requesting it.

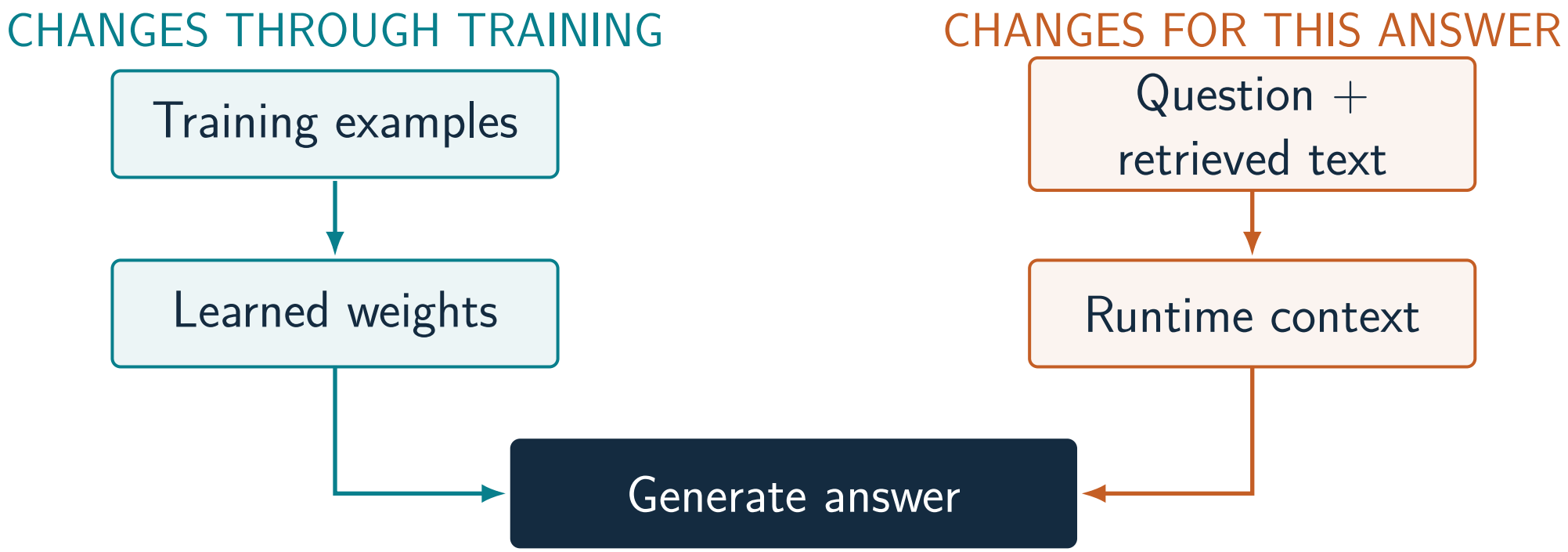
Why this matters: A prompt gives instructions; application integration makes tool execution possible.

Start with the language-model boundary — 3 of 3

1. The model receives an input	Instructions, the question and any supplied evidence are assembled into what the model can read.
2. The model can request a tool	A request names a tool and its arguments. The model has not executed the function merely by requesting it.
3. The application executes permitted requests	Python or a connected service runs the function and returns its result. The model then continues from that result.

Why this matters: A prompt gives instructions; application integration makes tool execution possible.

Two ways information reaches a model



Example: retrieve a newly signed contract today; train only when you want to change learned behaviour.

Different models can do different jobs — 1 of 3

1. Generator: write or request	A generative language model, such as GPT or Qwen, produces an answer or a structured tool request.

Why this matters: One application can use all three. That alone does not make its workflow agentic.

Different models can do different jobs — 2 of 3

1. Generator: write or request	A generative language model, such as GPT or Qwen, produces an answer or a structured tool request.
2. Embedding model: help find candidates	It converts a query and passages into vectors that a search service compares. OpenAI embedding models are one example.

Why this matters: One application can use all three. That alone does not make its workflow agentic.

Different models can do different jobs — 3 of 3

1. Generator: write or request	A generative language model, such as GPT or Qwen, produces an answer or a structured tool request.
2. Embedding model: help find candidates	It converts a query and passages into vectors that a search service compares. OpenAI embedding models are one example.
3. Reranker: compare the shortlist	A cross-encoder, such as a BGE reranker, scores each query–passage pair. It reorders candidates before generation.

Why this matters: One application can use all three. That alone does not make its workflow agentic.

Separate three architecture choices

Choice	Options	Question to ask
Model	General model or specialised model; smaller or larger.	What capability does this task need?
Hosting	Managed API or controlled deployment.	Who operates it and what policies apply?
Control flow	Fixed steps or agent-selected steps.	Which choices should the model make?

A small model can be hosted by a provider. An open-weight model can run in the cloud.

These are separate decisions, not mutually exclusive categories.

Choose the operating limitations first

Deployment	Main benefit	Main responsibility
Managed API	Provider operates serving and infrastructure.	Check service terms, data-policy compatibility and model changes.
Controlled deployment	Pin weights and manage the runtime.	Operate hardware, access controls, updates and monitoring.
Compact model for a bounded task	Potentially lower memory and latency.	Test task coverage and unfamiliar-case handling.

Also compare: cost, latency, reliability, context length, hardware and operating skills.

Example: a new contract motivates retrieval

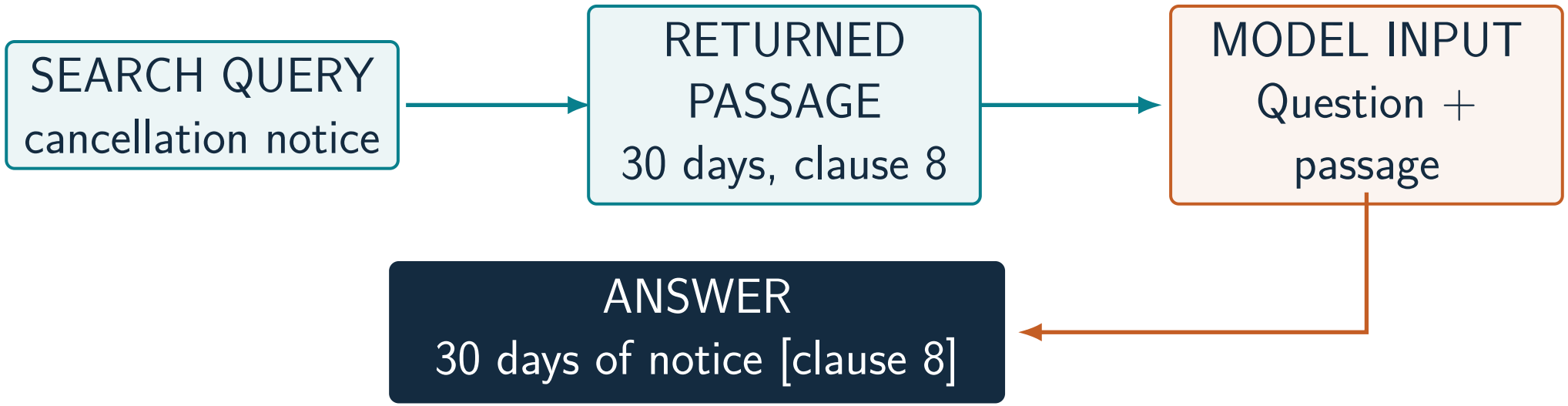


The learned weights may not contain todayś contract.

Practical response: retrieve the current contract and supply its text with the question.

Further training is a separate process for changing learned behaviour.

RAG makes a source part of the model input



The model receives selected text,
not automatic access to the whole library.

See the actual context supplied to the model

Instruction

Answer using the supplied passage. Cite its source.

Question

What is the cancellation notice period?

Retrieved context

[Contract v2, clause 8] Cancellation requires 30 days of written notice.

Generated answer: 30 days of written notice [Contract v2, clause 8].

RAG is not the same as vector search — 1 of 3

1. Retrieval finds information	A search tool can return text, table rows or API results. The application chooses and connects the permitted sources.

Why this matters: A SQL lookup followed by a generated explanation can also be retrieval-augmented generation.

RAG is not the same as vector search — 2 of 3

1. Retrieval finds information	A search tool can return text, table rows or API results. The application chooses and connects the permitted sources.
2. Augmentation puts results in context	The application supplies selected results with the original question. Being in a database is not the same as reaching the model.

Why this matters: A SQL lookup followed by a generated explanation can also be retrieval-augmented generation.

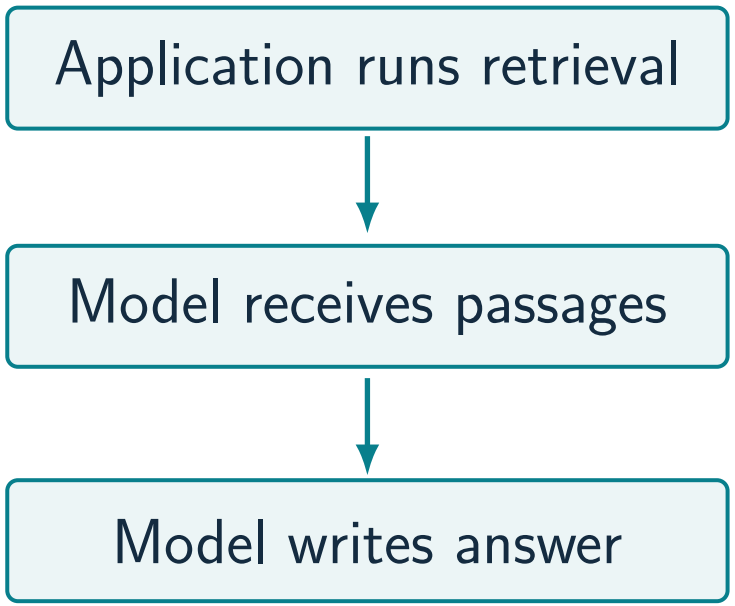
RAG is not the same as vector search — 3 of 3

1. Retrieval finds information	A search tool can return text, table rows or API results. The application chooses and connects the permitted sources.
2. Augmentation puts results in context	The application supplies selected results with the original question. Being in a database is not the same as reaching the model.
3. Generation produces the answer	The language model interprets the supplied context. Vector search is one possible retrieval method, not the whole RAG workflow.

Why this matters: A SQL lookup followed by a generated explanation can also be retrieval-augmented generation.

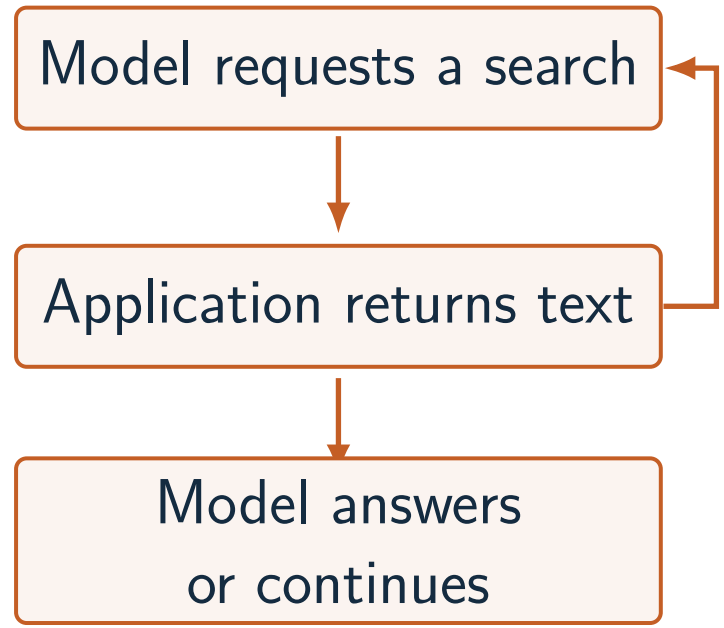
Fixed RAG versus agentic RAG

FIXED RAG



Code fixes the sequence.

AGENTIC RAG



The result can trigger another search.

Which component chooses the next step? — 1 of 3

1. Fixed sequence: code chooses	The application always retrieves, calculates and then asks the model to interpret. The model does not choose those earlier steps.

Why this matters: Inspect the trace: a recommendation to search does not execute a search.

Which component chooses the next step? — 2 of 3

1. Fixed sequence: code chooses	The application always retrieves, calculates and then asks the model to interpret. The model does not choose those earlier steps.
2. Agent loop: the model requests	The model sees a result and can request a follow-up search or calculation. The application executes each permitted request.

Why this matters: Inspect the trace: a recommendation to search does not execute a search.

Which component chooses the next step? — 3 of 3

1. Fixed sequence: code chooses	The application always retrieves, calculates and then asks the model to interpret. The model does not choose those earlier steps.
2. Agent loop: the model requests	The model sees a result and can request a follow-up search or calculation. The application executes each permitted request.
3. Either can follow an annex reference	Code can include a programmed branch. The choice is prescribed branching versus model-selected next steps, not tools versus no tools.

Why this matters: Inspect the trace: a recommendation to search does not execute a search.

Question: is this workflow agentic?

Code always retrieves a contract, adds the passage to the prompt, and asks the model to answer. What controls the steps?

- A. Application code controls a fixed RAG workflow.
- B. The model controls an agentic workflow because it writes the answer.
- C. The search index controls the application.

Answer: the application controls this sequence

Code always retrieves a contract, adds the passage to the prompt, and asks the model to answer. What controls the steps?

A. Application code controls a fixed RAG workflow.

The model generates the answer, but code chose the retrieval step and its place in the sequence.

An agentic version would let the model request additional permitted steps after observing results.

Harbor Metals: introduce the credit question

Fictional commodity company. Can its covenant headroom be called comfortable?

Financial filing	Loan agreement and annex
Debt: $75m; total cash: $15m.	Ceiling: 3.5 times net leverage.
EBITDA: $20m.	Exclude restricted cash identified in Annex A.
Downside EBITDA scenario: $17m.	Annex A: $5m restricted cash.

The filing supplies amounts; the contract defines which cash is eligible.

The retrieval output must reach the model

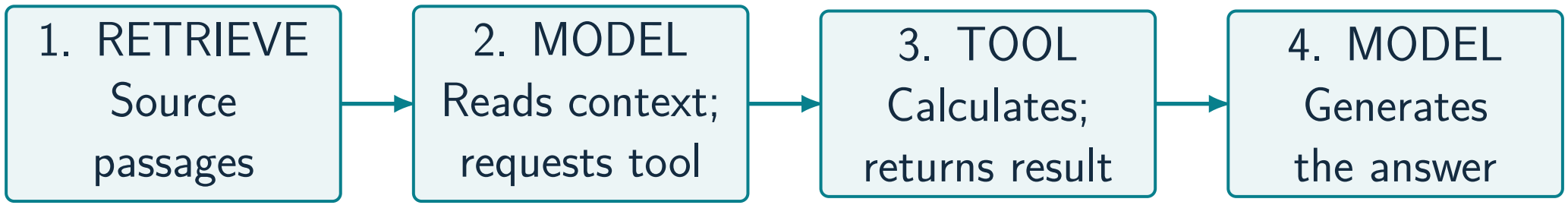
Retrieved source	Passage or record supplied in context
Filing	Debt 75; cash 15; EBITDA 20. Amounts are in millions.
Covenant	Net debt excludes restricted cash. Refer to Annex A.
Annex A	Restricted cash is 5 million.

Derived input: eligible cash $= 15 - 5 = 10$.

A document being in the library does not mean its passage was returned or supplied to the model.

Where retrieval and calculation fit

RAG + calculator: the application supplies source text to the model.



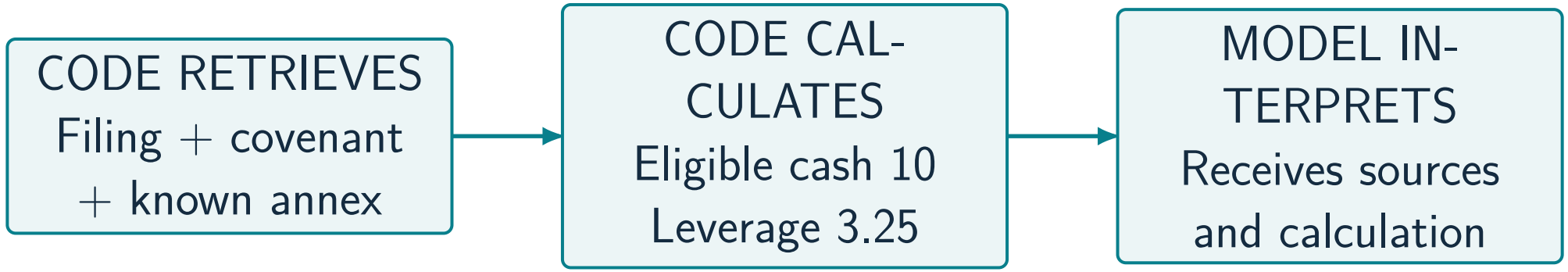
Source text: debt 75; eligible cash 10; EBITDA 20.

```
calculate_leverage(debt=75, eligible_cash=10, ebitda=20)
```

Returned result: 3.25×. The model uses it in its answer.

RAG names the overall retrieve-and-answer pattern, not a separate component.

Fixed workflow: code specifies the required steps



Model input: the credit question + selected passages + the calculator result.

The model need not choose a search query or request a calculator call in this workflow.

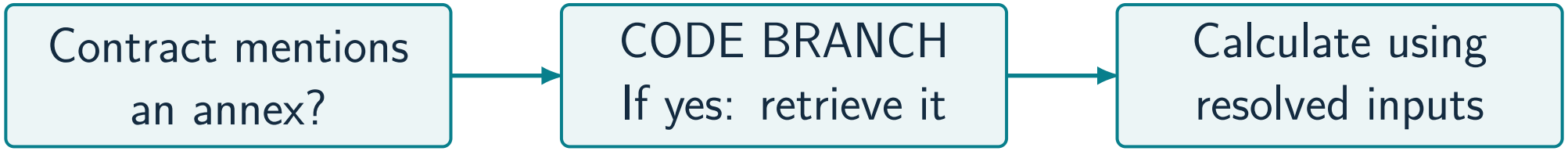
Agent workflow: the result changes the next request

1. Model requests the filing and covenant.
2. Returned clause says: Exclude restricted cash; see Annex A.
3. Model requests Annex A, then calculates with eligible cash 10.

The extra request follows from an observation.

Application code still checks, executes and records every permitted call.

Fixed code can follow a programmed branch too



Same evidence and calculation, different control.

An agent can choose a new request; fixed code can implement known conditional paths.

Neither approach can establish an unavailable annexś contents.

Harbor Metals: connect the model to two functions

Application exposes tools and their argument definitions.

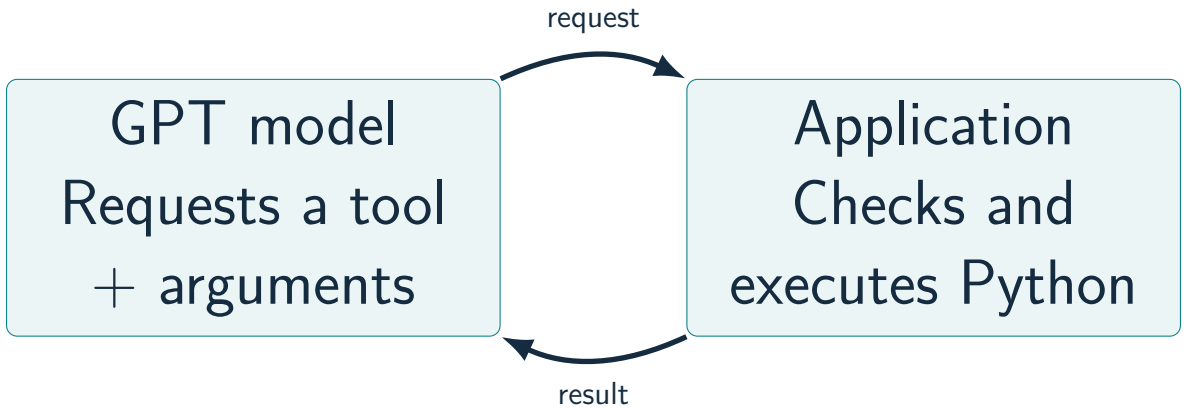
`search_documents(query)`
Returns cited filing, covenant or annex passages.

`calculate_leverage(debt, eligible_cash, ebitda)`
Returns the independently calculated leverage ratio.

Prompt: Assess covenant headroom. Follow annex references; use the calculator; cite sources.

The prompt gives instructions. Application integration makes tools available.

Direct function calling: request, execute, return

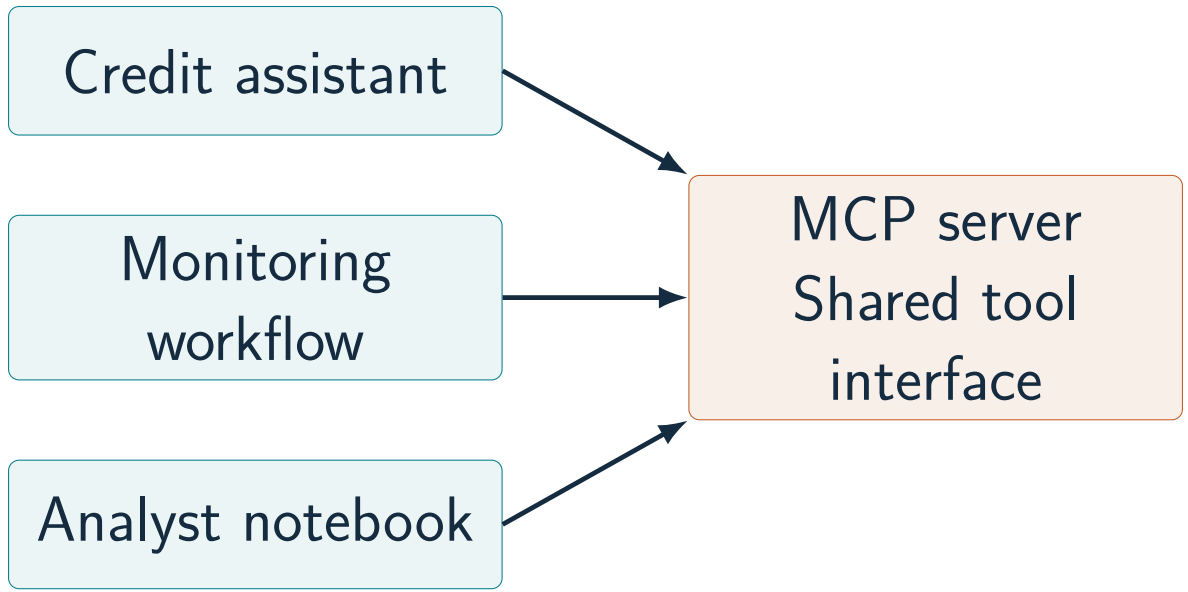


Illustrative trace: search filing → find annex reference → search annex → calculate → answer.

Eligible cash = 15 – 5 = 10; leverage = (75 – 10)/20 = 3.25×.

No MCP server is required for this connection.

When MCP helps: reuse Harbor Metals tools



Same search and calculator functions; different workflows.

MCP standardizes tool discovery and calls. It does not choose the workflow or grant authority.

Permission to calculate is not permission to release

Role	Search / calculate	Release brief
Research	Allowed	Denied
Reviewer	Allowed	Allowed while grant is valid
Revoked reviewer	Allowed	Denied

Enforcement: check the authenticated identity and current grant before execution. Record the result.

A prompt saying “do not release” is not a permission boundary.

Later: identity, revocation and accountable approval.

Class exercise: Harbor Metals tools and permissions

Open: `01b_harbor_tools_mcp_permissions.ipynb`

- Run C01–C05: search, follow ANNEX_A, calculate 3.25×.
- Leave RUN_LIVE and RUN_MCP False for the default path.
- Run C08: research denied; reviewer permitted; revoked reviewer denied.
- Inspect the AUDIT record and confirm only one simulated release.

Explain: connectivity, control flow and permission are separate.

Retrieval and training solve different problems — 1 of 3

1. A required fact is missing	Obtain the current source and supply it as context. Training on old examples cannot establish what an unavailable new document says.

Why this matters: First diagnose missing information versus recurring interpretation failure.

Retrieval and training solve different problems — 2 of 3

1. A required fact is missing	Obtain the current source and supply it as context. Training on old examples cannot establish what an unavailable new document says.
2. The source is present but repeatedly misread	Improve instructions. If errors persist, consider fine-tuning on reviewed demonstrations of the required behavior.

Why this matters: First diagnose missing information versus recurring interpretation failure.

Retrieval and training solve different problems — 3 of 3

1. A required fact is missing	Obtain the current source and supply it as context. Training on old examples cannot establish what an unavailable new document says.
2. The source is present but repeatedly misread	Improve instructions. If errors persist, consider fine-tuning on reviewed demonstrations of the required behavior.
3. An update appears to help	Evaluate on separate unseen cases. Better performance on the training examples alone does not establish generalization.

Why this matters: First diagnose missing information versus recurring interpretation failure.

Class exercise: inspect the handoffs

Open: `01_architectures.ipynb`

- Run S1, A1 and B1. Inspect returned sources and the cash argument 15.
- Run C1. Follow each requested tool and the eligible-cash argument 10.
- Run D1. The fixed annex branch also returns 3.25×.
- Remove ANNEX_A, rerun C1, then restore S1 and compare.

Explain: who selected each step, which evidence was supplied, and which calculation actually ran.

Question: what should change first?

An agent answers from an obsolete agreement. What should you investigate first?

- A. Which source version retrieval selected and supplied.
- B. Whether a larger model gives a different interpretation of that same obsolete text.
- C. Whether preference training makes the response sound more qualified.

Answer: inspect the selected source version

An agent answers from an obsolete agreement. What should you investigate first?

A. Which source version retrieval selected and supplied.

Check the returned source and the context sent to the model.

Changing the model does not establish which agreement is operative.

Section 2 examines these retrieval and source-selection failures.

Architecture choices involve practical trade-offs — 1 of 3

1. Check operating constraints	Data policy, security, permitted processing locations and retention requirements may rule out an otherwise capable service.

Why this matters: Compare designs on the same tasks, including failure handling and operational work.

Architecture choices involve practical trade-offs — 2 of 3

1. Check operating constraints	Data policy, security, permitted processing locations and retention requirements may rule out an otherwise capable service.
2. Measure the complete workload	Count provider calls, searches, calculations, retries and latency. Token price alone is not the cost of a completed task.

Why this matters: Compare designs on the same tasks, including failure handling and operational work.

Architecture choices involve practical trade-offs — 3 of 3

1. Check operating constraints	Data policy, security, permitted processing locations and retention requirements may rule out an otherwise capable service.
2. Measure the complete workload	Count provider calls, searches, calculations, retries and latency. Token price alone is not the cost of a completed task.
3. Match resources and skills	Local hosting needs hardware and operational support. Choose the simplest adequate workflow that meets the measured requirements.

Why this matters: Compare designs on the same tasks, including failure handling and operational work.

Section 1: objectives revisited

- **Model boundary:** identify the model input and output.
- **Information:** distinguish learned weights from supplied context.
- **Control:** explain which component selects and executes each step.
- **RAG:** explain how retrieved text or records reach generation.
- **Next:** Section 2 examines how the information is found and extracted.

STAC AI BOOTCAMP NYC



Retrieval and RAG

Matthew Dixon, PhD, FRM
BeliefLens

21 October 2026 | Principles, algorithms and evaluation

matthew.dixon@belieflens.org

Why retrieval knowledge still matters

- An LLM can interpret text that you supply.
- Your system still determines which information it sees.
- Understanding retrieval helps locate missing, irrelevant or misleading context.

Decision: Know whether to change the evidence pipeline or the model.

Python notebook: `02_grounded_rag.ipynb`. Start with C01–C06; no model key required.

A roadmap driven by design questions

Question	Concepts we need
What information can be searched?	Collection, passage units, metadata and indexes.
How do we find candidates?	Lexical and learned representations.
Which candidates reach the model?	Hybrid retrieval, reranking and context budgets.
How do we know it works?	Grounding, component metrics and controlled tests.

Decision: Follow the information through the system before selecting tools.

Five capabilities to develop

- Specify the retrieval problem and eligible collection.
- Explain how lexical and dense ranking work.
- Choose a retrieval and reranking architecture.
- Construct a bounded, inspectable model context.
- Evaluate retrieval and generation separately.

Decision: Turn each capability into an observable design or diagnostic task.

Define the problem before the scoring function — 1 of 3

1. Start with the information need	Specify what must be established: for example, the current cancellation notice period for a particular agreement.

Why this matters: The task defines relevance; a retrieval model estimates which items may satisfy it.

Define the problem before the scoring function — 2 of 3

1. Start with the information need	Specify what must be established: for example, the current cancellation notice period for a particular agreement.
2. Construct a search query	The application or model expresses that need as search text. Preserve the entity, date and other constraints.

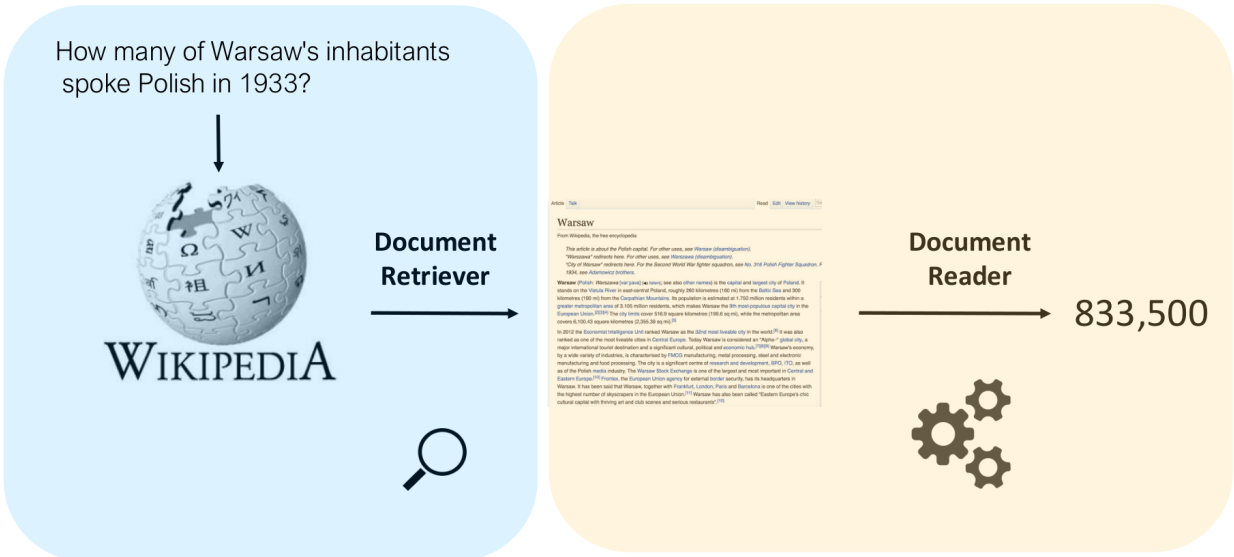
Why this matters: The task defines relevance; a retrieval model estimates which items may satisfy it.

Define the problem before the scoring function — 3 of 3

1. Start with the information need	Specify what must be established: for example, the current cancellation notice period for a particular agreement.
2. Construct a search query	The application or model expresses that need as search text. Preserve the entity, date and other constraints.
3. Judge relevance against the need	A passage is useful if it helps answer that scoped question. A high score alone does not make an old clause applicable.

Why this matters: The task defines relevance; a retrieval model estimates which items may satisfy it.

The retriever and reader solve different problems



Retriever

Select passages from a collection.

Reader / generator

Answer using the selected passages and question.

Measure both stages.

Decision: A retrieval miss and a reading error require different repairs.

Build once; retrieve many times

Collection preparation	Per-query work
Acquire and parse source versions.	Interpret the question and apply access scope.
Choose passage boundaries and preserve metadata.	Retrieve candidates and rerank.
Build lexical and/or vector indexes.	Select context and generate an answer.
Track additions, changes and deletions.	Record the source versions actually used.

Decision: Separate ingestion failures from query-time failures.

Choose what one searchable item means

Retrieval unit	Trade-off
Whole document	More context; a relevant section can be diluted.
Passage or section	More focused; may lose dependencies outside the boundary.
Sentence or field	Precise; can omit qualifications and referents.

Decision: Choose a unit that preserves the information needed for the target questions.

Chunking trades focus against context

- Small chunks improve focus but may split a condition from its conclusion.
- Large chunks retain context but consume budget and include unrelated text.
- Overlap reduces boundary loss while increasing duplicate content.
- Evaluate boundaries using answer-bearing spans and their qualifications.

Decision: Test chunking against information loss, not a universal token count.

Filters define eligibility; scores order candidates — 1 of 3

1. Check whether a source may be used	Access, scope and version rules determine eligibility. An old or restricted source cannot become eligible through a high score.

Why this matters: A relevant-looking passage still needs the right source, time and access status.

Filters define eligibility; scores order candidates — 2 of 3

1. Check whether a source may be used	Access, scope and version rules determine eligibility. An old or restricted source cannot become eligible through a high score.
2. Rank the eligible candidates	Keyword, vector or reranker scores prioritize potentially useful items. Ranking is separate from the hard source rules.

Why this matters: A relevant-looking passage still needs the right source, time and access status.

Filters define eligibility; scores order candidates — 3 of 3

1. Check whether a source may be used	Access, scope and version rules determine eligibility. An old or restricted source cannot become eligible through a high score.
2. Rank the eligible candidates	Keyword, vector or reranker scores prioritize potentially useful items. Ranking is separate from the hard source rules.
3. Keep uncertainty visible	If a version or permission is unknown, record the gap. Do not silently convert missing metadata into approval.

Why this matters: A relevant-looking passage still needs the right source, time and access status.

Notebook exercise: C02–C06. Inspect a passage, change the query, apply issuer/time filters.

Lexical retrieval depends on text representation

- Tokenization decides what counts as a term.
- Normalization can join variants but can also erase meaningful distinctions.
- Phrase and positional indexes preserve relationships beyond a bag of words.
- Negation, identifiers and punctuation may carry task-critical meaning.

Decision: Inspect the analyzer before blaming the ranking algorithm.

An inverted index avoids reading every document

Term	Documents containing the term
solar	D1, D3
battery	D2, D3
storage	D2

Query `solar AND battery`: intersect the lists to obtain D3.

Decision: Understand why exact-term retrieval is efficient and inspectable.

TF–IDF weights informative matches — 1 of 3

1. Count matching terms	Term frequency records how strongly a word appears in a passage. The query and passage use the same analyzed vocabulary.

Why this matters: Illustration: with the displayed unsmoothed IDF, a term in every document receives zero IDF.

TF–IDF weights informative matches — 2 of 3

1. Count matching terms	Term frequency records how strongly a word appears in a passage. The query and passage use the same analyzed vocabulary.
2. Give rare terms more discrimination	A word in every document helps little. A rare identifier can distinguish the requested item from many otherwise similar passages.

Why this matters: Illustration: with the displayed unsmoothed IDF, a term in every document receives zero IDF.

TF–IDF weights informative matches — 3 of 3

1. Count matching terms	Term frequency records how strongly a word appears in a passage. The query and passage use the same analyzed vocabulary.
2. Give rare terms more discrimination	A word in every document helps little. A rare identifier can distinguish the requested item from many otherwise similar passages.
3. Compare weighted representations	Cosine compares query and passage vectors with length normalization. The result is a ranking score, not a truth probability.

Why this matters: Illustration: with the displayed unsmoothed IDF, a term in every document receives zero IDF.

TF–IDF: read the score and its assumptions

$$w_{t,d} = \mathrm{tf}(t,d) \log \frac{N}{\mathrm{df}(t)}, \qquad s(q,d) = \frac{\mathbf{w}_q^\top \mathbf{w}_d}{\|\mathbf{w}_q\| \, \|\mathbf{w}_d\|}$$

t: term; d: document; q: query; N: collection size. $\mathrm{tf}(t,d)$: term count; $\mathrm{df}(t)$: documents containing that term.

$w_{t,d}$: term weight; $\mathbf{w}_q, \mathbf{w}_d$: weighted vectors. $\top$: transpose; $\|\cdot\|$: Euclidean length; s: cosine similarity.

The displayed IDF is unsmoothed and requires $\mathrm{df}(t) > 0$. The notebook uses smoothed IDF $\log((N+1)/(\mathrm{df}+1)) + 1$. Both rank lexical matches, not truth.

BM25 limits the benefit of repeated words — 1 of 3

1. Repetition should have diminishing value	Mentioning a query word one hundred times should not automatically beat a short paragraph that directly answers the question.

Why this matters: BM25 is a transparent lexical baseline; it does not resolve every synonym or qualification.

BM25 limits the benefit of repeated words — 2 of 3

1. Repetition should have diminishing value	Mentioning a query word one hundred times should not automatically beat a short paragraph that directly answers the question.
2. Account for document length	Long documents naturally contain more words. BM25 adjusts frequency relative to document length and the collection average.

Why this matters: BM25 is a transparent lexical baseline; it does not resolve every synonym or qualification.

BM25 limits the benefit of repeated words — 3 of 3

1. Repetition should have diminishing value	Mentioning a query word one hundred times should not automatically beat a short paragraph that directly answers the question.
2. Account for document length	Long documents naturally contain more words. BM25 adjusts frequency relative to document length and the collection average.
3. Tune two explicit choices	k1 controls frequency saturation; b controls length adjustment. Validate both on representative queries rather than treating a score as confidence.

Why this matters: BM25 is a transparent lexical baseline; it does not resolve every synonym or qualification.

BM25: read the score and its assumptions

$$s(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \frac{f_{t,d}(k_1+1)}{f_{t,d} + k_1(1 - b + b|d|/\overline{L})}$$

q: query; d: document; t: query term. $\Sigma_{t \in q}$: add term contributions. $\mathrm{IDF}(t)$: chosen rarity weight.

$f_{t,d}$: term frequency; $|d|$: document length; $\overline{L}$: average collection length.

$k_1 > 0$: frequency saturation; $b \in [0,1]$: length adjustment. $s(q,d)$: ranking score.

Repetition has diminishing benefit. The notebook uses TF–IDF cosine, not BM25.

Question: can lexical search miss the right passage?

A useful passage expresses the same idea with different words. Which conclusion follows?

- A. Retrieval is impossible.
- B. Exact term matching may miss it; expansion or learned representations may help.
- C. Any dense retriever is guaranteed to recover it.

Decision: Choose a remedy that addresses the observed vocabulary mismatch.

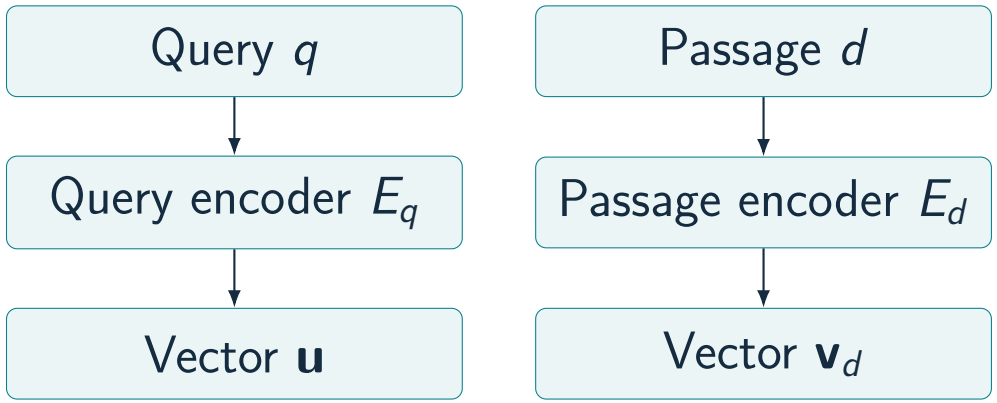
Answer: test a representation that bridges the wording

B. Expansion or learned representations may help.

- Keep exact-term retrieval as a baseline.
- Test paraphrases alongside entity, number and negation distinctions.
- Measure both recovered relevant passages and newly introduced errors.

Decision: A method is useful when it fixes a measured failure at acceptable cost.

A bi-encoder makes passage vectors reusable



$$\mathbf{u} = E_q(q), \quad \mathbf{v}_d = E_d(d)$$

$$s(q, d) = \mathbf{u}^\top \mathbf{v}_d$$

q: query text; d: passage text. E_q, E_d: learned encoders. $\mathbf{u}, \mathbf{v}_d$: vectors in the same space.

s: similarity; $\top$: transpose, making a dot product. Store passage vectors; encode each new query.

Optional notebook C07: actual local embeddings; default skips the model download.

Dense retrieval learns which passages belong together — 1 of 3

1. Supply a labelled comparison	Record: query=current meeting day; positive=the Friday update; negative=the superseded Tuesday notice. These are relevance labels.

Why this matters: The training set needs useful near-misses, not only unrelated passages.

Dense retrieval learns which passages belong together — 2 of 3

1. Supply a labelled comparison	Record: query=current meeting day; positive=the Friday update; negative=the superseded Tuesday notice. These are relevance labels.
2. Train relative similarity	The loss encourages the positive passage to score above the sampled negatives. The examples define which distinctions the representation learns.

Why this matters: The training set needs useful near-misses, not only unrelated passages.

Dense retrieval learns which passages belong together — 3 of 3

1. Supply a labelled comparison	Record: query=current meeting day; positive=the Friday update; negative=the superseded Tuesday notice. These are relevance labels.
2. Train relative similarity	The loss encourages the positive passage to score above the sampled negatives. The examples define which distinctions the representation learns.
3. Test the learned distinctions	False negatives, date changes and domain mismatch can cause errors. Nearby vectors do not by themselves prove relevance or truth.

Why this matters: The training set needs useful near-misses, not only unrelated passages.

Dense retrieval: read the loss and its assumptions

$$\mathcal{L} = -\log \frac{\exp(s(q, d^+)/\tau)}{\exp(s(q, d^+)/\tau) + \Sigma_j \exp(s(q, d_j^-)/\tau)}$$

q: query; d^+: labelled relevant passage; d_j^-: negative passage j. s: similarity score; $\tau > 0$: temperature.

exp: exponential; log: natural logarithm; Σ_j: sum over sampled negatives. $\mathcal{L}$: loss minimized during training.

The positive competes against this sampled set. Lower loss rewards its relative score; incorrect negatives teach the wrong distinction.

The optional notebook loads an existing encoder; it does not train this loss.

Negative examples determine the difficult distinctions

Negative type	What it teaches / risks
Random unrelated passage	Broad topic discrimination; may be too easy.
High-ranked but irrelevant passage	Fine distinctions; requires careful labels.
Relevant passage wrongly labelled negative	Incorrectly pushes useful evidence away.

Decision: Curate near-misses that resemble real retrieval failures.

Approximate search introduces a second recall question

Query vector ⟶ vector index ⟶ candidate IDs

- An exact scan compares the query with every vector.
- Approximate nearest-neighbour search trades speed and memory against exact top-result recovery.
- An exact nearest neighbour can still be irrelevant to the task.

Decision: Measure index recall separately from task relevance recall.

Hybrid retrieval combines different candidate signals

$$\mathrm{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{c + \mathrm{rank}_r(d)}$$

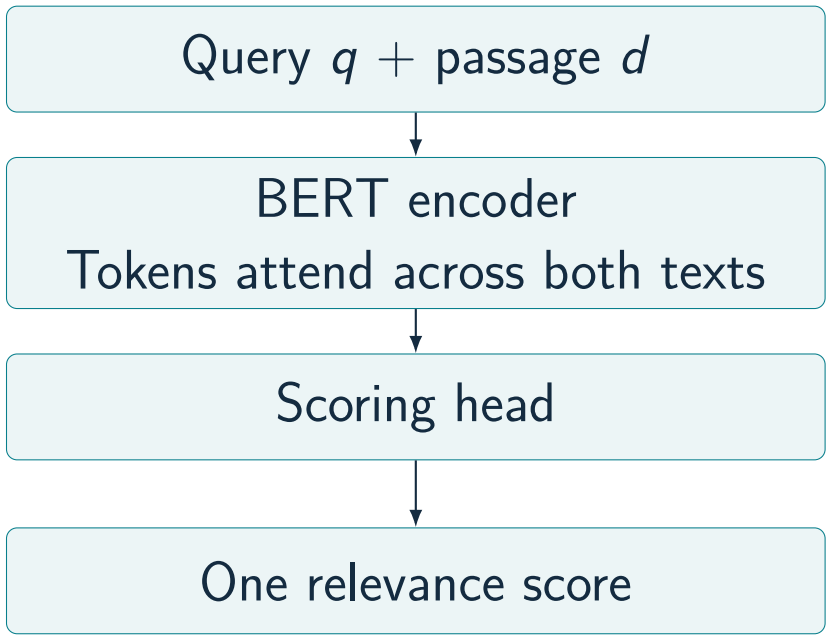
Reciprocal rank fusion: combine list positions instead of adding unlike raw scores.

d: candidate document; $\mathcal{R}$: set of ranked lists; r: one list. $\mathrm{rank}_r(d)$: its position, starting at 1.

$c > 0$: smoothing constant; Σ: add contributions from lists containing d. A missing document contributes zero.

Example with $c = 60$: ranks 1 and 3 give $1/61 + 1/63 \approx 0.0323$. This is a ranking score, not probability.

A cross-encoder reads the query and passage jointly



BERT: Bidirectional Encoder Representations from Transformers. It reads both left and right context within the supplied pair.

$$s_{\text{cross}}(q, d) = f_\theta([q; d])$$

$[q; d]$: query and passage joined in one input.
f_θ: learned scoring model; θ: its parameters.
s_{cross}: pair score.

Run once per candidate pair; use a shortlist.
The score is not automatically calibrated.

The shortlist creates a ceiling for reranking — 1 of 3

1. First-stage search creates candidates	If the required passage is not in that set, it is unavailable to a pure reranker.

Why this matters: Do not replace the generator to repair evidence that never reached it.

The shortlist creates a ceiling for reranking — 2 of 3

1. First-stage search creates candidates	If the required passage is not in that set, it is unavailable to a pure reranker.
2. Reranking orders what it receives	A stronger comparison model can promote a useful candidate. It cannot introduce a passage absent from its input.

Why this matters: Do not replace the generator to repair evidence that never reached it.

The shortlist creates a ceiling for reranking — 3 of 3

1. First-stage search creates candidates	If the required passage is not in that set, it is unavailable to a pure reranker.
2. Reranking orders what it receives	A stronger comparison model can promote a useful candidate. It cannot introduce a passage absent from its input.
3. Repair the earliest failed stage	Check collection and retrieval for a missing candidate; check reranking for a low-ranked candidate; check interpretation if context is already correct.

Why this matters: Do not replace the generator to repair evidence that never reached it.

Query rewriting helps only if intent is preserved

- Rewrite vague wording or expand useful synonyms.
- Split a compound information need into explicit subqueries.
- Retain entities, dates, negation and constraints.
- Record the original query, rewrite and returned evidence.

Decision: Evaluate rewrites as interventions, not hidden preprocessing.

Multiple retrieval steps require explicit stopping rules

Query → passages → identified gap → follow-up query

- A follow-up should address a named missing fact or relationship.
- Bound the number of queries, latency and spending.
- Stop with an unresolved gap when no eligible support is found.

Decision: Use iterative retrieval when later information needs depend on earlier results.

Context construction is a selection problem — 1 of 3

1. Search output is not yet model input	The application chooses which candidate passages to supply. The generator cannot read candidates omitted from its context.

Why this matters: A larger context window is capacity, not proof that every useful passage was selected or used.

Context construction is a selection problem — 2 of 3

1. Search output is not yet model input	The application chooses which candidate passages to supply. The generator cannot read candidates omitted from its context.
2. Spend the input budget deliberately	Keep controlling clauses, exceptions and source IDs. Duplicates and obsolete versions can crowd out essential information.

Why this matters: A larger context window is capacity, not proof that every useful passage was selected or used.

Context construction is a selection problem — 3 of 3

1. Search output is not yet model input	The application chooses which candidate passages to supply. The generator cannot read candidates omitted from its context.
2. Spend the input budget deliberately	Keep controlling clauses, exceptions and source IDs. Duplicates and obsolete versions can crowd out essential information.
3. Inspect the assembled input	Count actual tokens including instructions and wrappers. Save the selected text and tool results, not only the search list.

Why this matters: A larger context window is capacity, not proof that every useful passage was selected or used.

Historical RAG-Sequence: marginalizing candidate documents

Historical RAG-Sequence architecture: mix answer likelihoods over a retrieved document variable.

$$p(y \mid q) \approx \sum_{d \in \mathrm{TopK}(q)} p_\eta(d \mid q)\, p_\theta(y \mid q, d)$$

q: query; y: complete answer token sequence; d: candidate document.

$\mathrm{TopK}(q)$: selected K documents; Σ: add their contributions.

η: retriever parameters; θ: generator parameters. Each term multiplies a retrieval weight by an answer likelihood given that document.

Practical distinction: many RAG services concatenate passages into one prompt. They do not compute this mixture. Neither approach automatically gives a calibrated probability of truth.

Make the actual model input inspectable

Input component	Inspection question
Task and original question	What answer is being requested?
Selected source text and IDs	What evidence can the model actually see?
Instructions for uncertainty and citations	What response behavior is required?
Tool outputs, if any	Which computations were actually executed?

Decision: Store the assembled context, not only search results.

Notebook exercise: C08–C09. Read the three queries and the exact assembled prompt.

Retrieval can bring instructions as well as evidence

- Source text may contain commands addressed to a reader or model.
- Treat retrieved material as data within the intended task.
- Keep tool permissions and action checks outside source-controlled content.
- Test hostile or misleading passages as part of evaluation.

Decision: Do not let a source redefine the task or grant authority.

Support is different from citation presence

Check	What it establishes
Source exists	The cited record can be found.
Quoted text matches	The words appear in the source.
Claim is supported	The passage establishes the claim with its qualifications.
Source is reliable and applicable	A separate source-quality and scope judgement.

Decision: Evaluate citation correctness and claim support independently.

Evidence graphs organize declared dependencies

Source span → extracted fact → computation → claim

- Record which inputs each claim or calculation requires.
- Re-evaluate affected descendants when a source changes or disappears.
- Treat the graph as a record of declared dependencies; test whether those links are justified.

Decision: Use a graph to localize missing support and review impact.

Notebook exercise: C10–C13. Remove ANNEX, inspect the blocked path, then restore it.

Evaluate the stages with a fixed test collection

You need	Why
Representative queries and information needs	Define the target population of tasks.
Relevant passages and no-answer cases	Measure misses and unsupported completion.
Held-out queries and fixed corpus versions	Separate tuning from evaluation.
Recorded configurations and context	Reproduce and compare pipeline behavior.

Decision: Build a test collection before optimizing the retrieval stack.

Precision and recall measure different costs — 1 of 3

1. Count the relevant returned items	Suppose two of three returned passages are relevant, and four relevant passages exist in the labelled collection.

Why this matters: These are retrieval-list measures. Evaluate the generated answer separately.

Precision and recall measure different costs — 2 of 3

1. Count the relevant returned items	Suppose two of three returned passages are relevant, and four relevant passages exist in the labelled collection.
2. Precision asks how clean the list is	Two useful results out of three returned gives precision 2/3. Irrelevant results can distract the generator.

Why this matters: These are retrieval-list measures. Evaluate the generated answer separately.

Precision and recall measure different costs — 3 of 3

1. Count the relevant returned items	Suppose two of three returned passages are relevant, and four relevant passages exist in the labelled collection.
2. Precision asks how clean the list is	Two useful results out of three returned gives precision 2/3. Irrelevant results can distract the generator.
3. Recall asks how much was recovered	Two found out of four labelled relevant items gives recall 1/2. A missing exception may matter even with high precision.

Why this matters: These are retrieval-list measures. Evaluate the generated answer separately.

Precision and recall: read the denominators

$$\text{Precision@k} = \frac{|R_k \cap G|}{k}, \qquad \text{Recall@k} = \frac{|R_k \cap G|}{|G|}$$

k: requested cutoff; R_k: returned distinct top-k items; G: labelled relevant items.

$\cap$: intersection; $|\cdot|$: number of items. The numerator counts returned relevant items.

Example: 2 relevant of 3 returned, with 4 relevant in the collection: precision $2/3$, recall $2/4$.

Specify empty-result handling. The notebook divides by requested k even if fewer than k results return.

Notebook exercise: C14. Inspect Retrieved, Recall and Missed; change only k.

MRR: how soon do we find the first useful result?

$$\mathrm{MRR} = \frac{1}{Q}\sum_{i=1}^{Q} \mathrm{RR}_i, \qquad \mathrm{RR}_i = \begin{cases} 1/r_i & \text{if a relevant result is found,} \\ 0 & \text{otherwise.} \end{cases}$$

Mean reciprocal rank (MRR): average reward for finding the first useful result early.

Q: number of evaluated queries; i: query index; r_i: rank of its first relevant result (starting at 1). RR_i: that query's reciprocal rank.

Ranks 1, 2 and no hit give $\mathrm{MRR} = (1 + 1/2 + 0)/3 = 0.50$.

Limit: later relevant passages do not change MRR once the first hit is fixed.

nDCG: reward several useful results in the right order

$$\mathrm{DCG@k} = \sum_{j=1}^{k} \frac{2^{g_j} - 1}{\log_2(j+1)}, \qquad \mathrm{nDCG@k} = \frac{\mathrm{DCG@k}}{\mathrm{IDCG@k}}$$

k: cutoff; j: rank; g_j: relevance grade at that rank. $2^{g_j} - 1$: gain; $\log_2(j+1)$: position discount.

DCG: discounted cumulative gain. **IDCG:** DCG for the ideal grade ordering. **nDCG:** normalized DCG, from 0 to 1 when ideal gain is positive.

Grades [0,2,1] give $0 + 3/\log_2 3 + 1/2 \approx 2.393$. Ideal [2,1,0] gives $3 + 1/\log_2 3 \approx 3.631$. Thus nDCG ≈ 0.659.

Declare grades, cutoff and handling of zero ideal gain. This measures ordering against labels, not answer correctness.

One small example makes the trade-off visible

Top three relevance labels	Precision@3	Recall@3
Relevant, irrelevant, relevant	2/3	2/4
Relevant, relevant, relevant	3/3	3/4

Assume four labelled relevant passages in the collection.

Neither result set establishes complete coverage.

Decision: Read the numerator and denominator before interpreting a metric.

Evaluate answers separately from the retrieval list

Dimension	Question
Correctness	Does the answer agree with the adjudicated target?
Groundedness	Are its claims supported by supplied evidence?
Citation quality	Do references support the linked claims?
Abstention	Does it acknowledge insufficient evidence?
Efficiency	What latency and cost were needed?

Decision: A single answer score cannot explain why a system failed.

Controlled comparisons reveal which stage matters

Intervention	Question it isolates
Supply adjudicated supporting passages	Can the reader answer with good evidence?
Hold the reader fixed; change retriever	Does candidate selection improve?
Hold passages fixed; change order	Is context use position-sensitive?
Remove a required passage	Does the system detect the evidence gap?

Decision: Change one component or evidence condition at a time.

Similarity and confidence require different validation — 1 of 3

1. A score orders candidates	Its value depends on the representation, model and candidate set. A cosine score of 0.9 is not 90 percent confidence in an answer.

Why this matters: Define the event before interpreting any score as a probability.

Similarity and confidence require different validation — 2 of 3

1. A score orders candidates	Its value depends on the representation, model and candidate set. A cosine score of 0.9 is not 90 percent confidence in an answer.
2. Normalization does not establish calibration	Numbers summing to one can still be poor estimates of a defined real-world event.

Why this matters: Define the event before interpreting any score as a probability.

Similarity and confidence require different validation — 3 of 3

1. A score orders candidates	Its value depends on the representation, model and candidate set. A cosine score of 0.9 is not 90 percent confidence in an answer.
2. Normalization does not establish calibration	Numbers summing to one can still be poor estimates of a defined real-world event.
3. Validate any decision threshold	Use reviewed cases to measure coverage and admitted errors. Recheck after source, model or task changes.

Why this matters: Define the event before interpreting any score as a probability.

Question: where should the repair begin?

A reranker never receives the required passage. Which change directly addresses this failure?

- A. Improve the collection or candidate retrieval.
- B. Ask the generator to sound more certain.
- C. Raise the reranker score threshold.

Decision: Locate the first missing dependency.

Answer: repair the candidate path first

A. Check collection coverage, then candidate retrieval.

- If the source is absent, acquire or index it.
- If it is indexed, inspect query representation and retrieval settings.
- Only then assess whether reranking and reading preserve the useful evidence.

Decision: Use the smallest intervention that addresses the observed failure.

Select the architecture from observed requirements

Observed requirement	Candidate design choice
Exact names, phrases and identifiers	Strong lexical baseline and appropriate analyzer.
Frequent paraphrases	Evaluate dense or hybrid retrieval.
Subtle query-passage compatibility	Evaluate a cross-encoder reranker.
Dependent information needs	Bounded iterative retrieval.
Strict historical or access constraints	Enforced filters and versioned source capture.

Decision: Choose components to fix measured failure modes under a cost budget.

The roadmap now becomes a diagnostic checklist

- Can the required evidence be found in the indexed collection?
- Does the query representation retrieve it?
- Do filters, reranking and context selection retain it?
- Does the reader use it with the right qualifications?
- Do held-out tests support the performance claim?

Decision: Explain a failure by stage, then design the next test.

Lecture sources: retrieval foundations

Material	Role in this lecture
Manning, Raghavan and Schuetze: Introduction to Information Retrieval	Chapters 1–2: indexing; 6: scoring; 8: evaluation; 9: expansion; 11: BM25.
Stanford CS224N 2025, Lecture 13	QA taxonomy, retriever–reader framework and retrieval models.
Karpukhin et al., EMNLP 2020	Dense passage encoders, training and negatives.

Decision: Use these references to deepen the mechanism you need to change.

Lecture sources: RAG and context use

Material	Role in this lecture
Sentence Transformers: Retrieve and Re-Rank	Bi-encoder / cross-encoder architecture and diagrams.
Lewis et al., NeurIPS 2020	Retrieval-conditioned generation and RAG-Sequence.
Liu et al., TACL 2024	Controlled context-position experiment.
Microsoft Azure AI Search documentation	Hybrid retrieval, rank fusion and filters.

Decision: Use primary results with their model, task and implementation limits.

Section 2: objectives revisited

Objective	What you should now be able to explain
Retrieval	Explain collection preparation, filtering and passage ranking.
Evidence quality	Distinguish eligibility, relevance and claim support.
Repair	Name the missing dependency and test a valid replacement.
Historical use	Identify which exact source version was available at the decision.

Break

11:15–11:30

STAC AI BOOTCAMP NYC



Post-training for financial applications

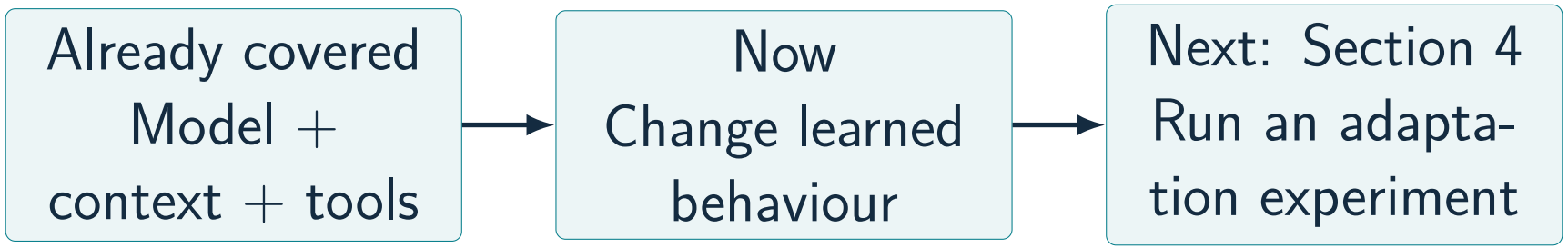
Matthew Dixon, PhD, FRM
BeliefLens

21 October 2026 | 11:30–12:30

matthew.dixon@belieflens.org

What changes in Section 3?

Notebook: `03_post_training.ipynb`



Why train? Better information does not automatically teach a model how to respond.

Post-training means additional training of an existing model.

Five things to be able to explain

- What an SFT demonstration teaches and which tokens are scored.
- Why LoRA is a parameter-update method, not a training objective.
- What rank and quantization change.
- How DPO and conventional RLHF use preference feedback.
- What independent evaluation must establish.

Start with three separate questions

Question	Options	What it controls
What should improve?	SFT; preference learning	The learning signal.
Which parameters can change?	Full tuning; LoRA	The space of trainable updates.
How is the model stored?	Higher precision; quantized	Memory and numerical representation.

These choices combine: SFT + LoRA; DPO + LoRA; SFT + QLoRA.

SFT: supervised fine-tuning — 1 of 3

1. Provide a reviewed demonstration	A record contains the task, input and response we want the model to learn. The target response is the label.

In plain English: Example: input The meeting is on Tuesday; reviewed completion Tuesday.

SFT: supervised fine-tuning — 2 of 3

1. Provide a reviewed demonstration	A record contains the task, input and response we want the model to learn. The target response is the label.
2. Reward the recorded answer tokens	Training increases their conditional likelihood. It does not automatically check whether the reviewer answer is true.

In plain English: Example: input The meeting is on Tuesday; reviewed completion Tuesday.

SFT: supervised fine-tuning — 3 of 3

1. Provide a reviewed demonstration	A record contains the task, input and response we want the model to learn. The target response is the label.
2. Reward the recorded answer tokens	Training increases their conditional likelihood. It does not automatically check whether the reviewer answer is true.
3. Test on different cases	A model can fit the demonstrations without handling new wording or completing the whole workflow correctly.

In plain English: Example: input The meeting is on Tuesday; reviewed completion Tuesday.

What one training record contains

Part	Example	Purpose
Instruction	Extract the meeting day.	Defines the task.
Input	The meeting is on Tuesday.	Provides the context.
Target response	Tuesday.	Provides the demonstration.

The chat template converts these parts into one token sequence.

Decide explicitly: which response positions contribute to loss?

Teacher forcing: predict each next target token



Training supplies the correct preceding answer tokens.

At generation time, the model instead continues from its own previous output.

A small labelled SFT dataset

Input record	Reviewed target label
Extract the day: The meeting is on Tuesday.	Tuesday.
Extract the current day: The meeting moved from Tuesday to Friday.	Friday.

The response is the label. In a prompt–completion dataset it is stored as the completion.

The examples teach a task, not a table of confidence scores.

The loss mask selects what is scored — 1 of 3

1. Keep the question visible	The model still reads the instruction and input while predicting the response.

In plain English: Changing a zero-loss prompt can still change the scored answer predictions.

The loss mask selects what is scored — 2 of 3

1. Keep the question visible	The model still reads the instruction and input while predicting the response.
2. Score the selected answer positions	A mask of one includes a prediction error. A mask of zero excludes that position from direct loss.

In plain English: Changing a zero-loss prompt can still change the scored answer predictions.

The loss mask selects what is scored — 3 of 3

1. Keep the question visible	The model still reads the instruction and input while predicting the response.
2. Score the selected answer positions	A mask of one includes a prediction error. A mask of zero excludes that position from direct loss.
3. Do not confuse this with attention	A loss mask selects errors to score. An attention mask controls which positions can interact.

In plain English: Changing a zero-loss prompt can still change the scored answer predictions.

Before the equation: what is assumed?

- The model predicts each token conditional on the preceding tokens; tokens are not assumed independent.
- Reviewed targets represent the behavior to imitate. Wrong labels teach the wrong behavior.
- This example uses teacher forcing and response-only scoring.
- The displayed reduction is a token-weighted average; other weighting conventions are possible.

SFT loss: one average over scored tokens

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\frac{1}{M} \sum_{i=1}^{N} \sum_{t=1}^{T_i} m_{i,t} \log p_\theta(z_{i,t} \mid z_{i,<t})$$

- θ: trained parameters; $\mathcal{L}_{\mathrm{SFT}}$: mean supervised loss; N: examples; i: example index; T_i: token count; t: token position.
- $z_{i,t}$: current token; $z_{i,<t}$: preceding tokens; p_θ: model probability; $|$: conditional on.
- $m_{i,t}$: mask, 1 if scored and 0 otherwise; $M = \Sigma_{i,t}\, m_{i,t} > 0$: total scored tokens; Σ: sum; log: natural logarithm.
- Read: average negative log probability of the demonstrated tokens. Loss is in nats per scored token; prompt masking does not remove context.

A tiny calculation makes the loss concrete

Scored token	Assigned probability	Negative log probability
First target token	0.8	0.223
Second target token	0.5	0.693

$$\text{Mean loss} = (0.223 + 0.693)/2 = 0.458$$

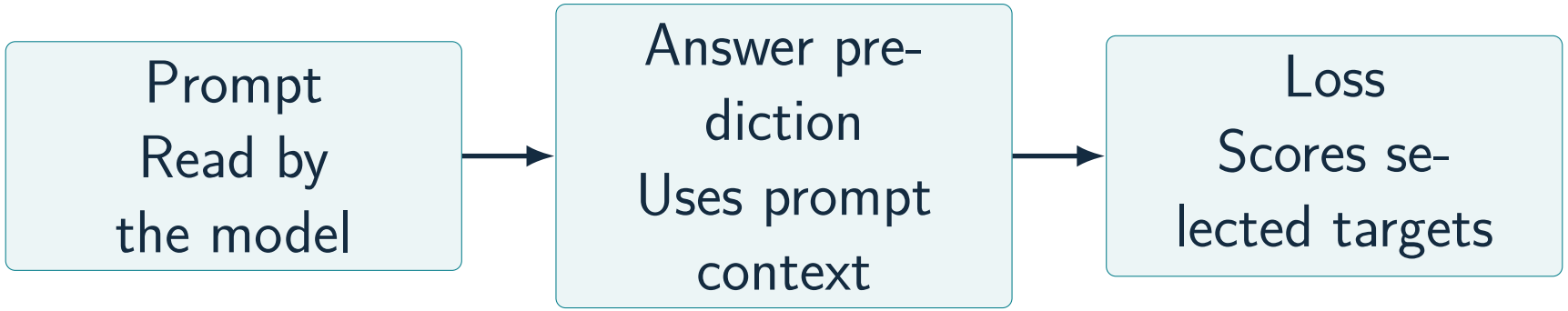
Higher probability on recorded targets lowers loss.

It does not establish success on unseen tasks.

Check 1: what does a zero loss mask mean?

- A. The model cannot read that position.
- B. That position has no direct contribution to the loss.
- C. The model assigns that token zero probability.

Answer: B — excluded from direct loss



The prompt remains available to help predict the response.

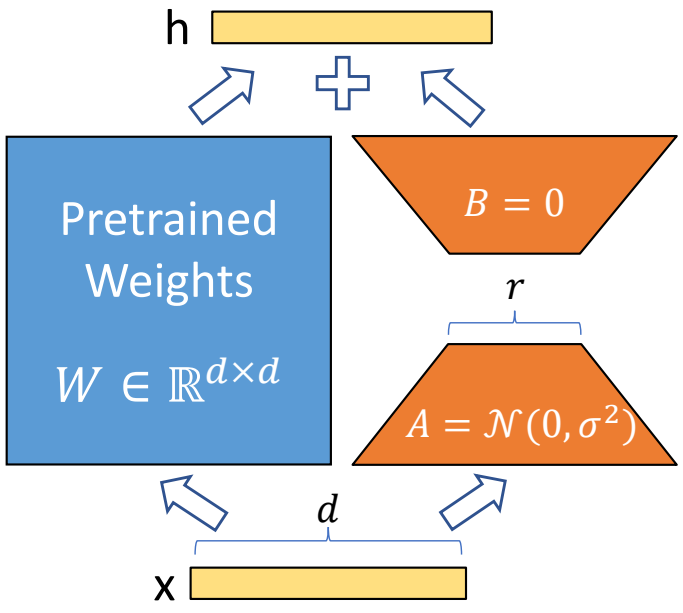
The loss mask only selects which prediction terms enter the objective.

Why not update every parameter?

- A large model contains billions of weights. Updating all of them needs gradients and optimizer memory.
- A separate full model for each task is also expensive to store and maintain.
- LoRA trains a small correction while keeping the original weights fixed.

Purpose: make task adaptation feasible with fewer trainable parameters.

LoRA: low-rank adaptation



Read the original figure

Blue: the original transformation stays fixed.

Orange: small matrices A and B learn a correction.

Both paths receive x; their outputs are added to produce h.

The added branch is inside a selected layer, not a separate model between layers.

x: input; h: output; W_0: frozen matrix; A, B: trainable correction matrices; r: positive rank; α: scale. The multiplier α/r is unitless.

LoRA equation: base transformation + correction

$$h = W_0 x + \frac{\alpha}{r} BAx$$

- x: input vector; h: output vector; W_0: frozen base matrix, with output-by-input shape.
- A: trainable input-to-rank matrix; B: trainable rank-to-output matrix; r: positive integer rank; α: scale; α/r: unitless multiplier.
- Read: add a learned low-rank correction to the same base transformation. Vector units depend on the representation; weights map between them.
- Only the selected adapter matrices are updated. This is a parallel branch inside a layer, not a smaller replacement model or a layer inserted between every pair of layers.

What changes when rank increases?

Rank for a 4096 by 4096 matrix	Added trainable parameters	Interpretation
4	32,768	Smaller update space.
8	65,536	Twice as many trainable entries.
16	131,072	More update capacity and resource use.

Larger rank permits a richer correction; it does not guarantee a better model.

Choose rank using held-out evaluation and the compute budget.

Who chooses where LoRA is applied?

Choice	Who determines it?
Target modules: Q, K, V, output or feed-forward projections	Developer configuration.
Rank and scaling	Developer configuration, evaluated experimentally.
Values inside A and B	Backpropagation and the optimizer.

```
LoraConfig(r=8, target_modules=["q_proj", "v_proj"])
```

Q/V is a choice, not a universal optimum. Compare alternatives on the task.

Why QLoRA? LoRA still keeps a large base in memory

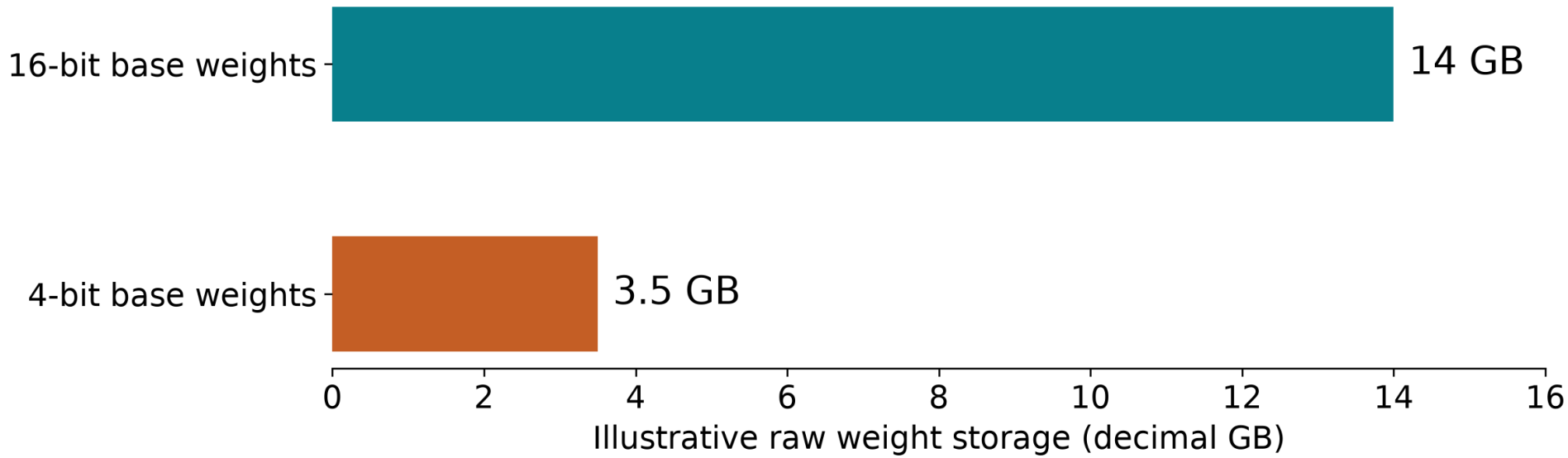


Illustration: 7 billion weights $\times$ 16 bits $=$ 14 GB; at 4 bits $=$ 3.5 GB.

Raw weights only: excludes scales, adapters, activations and runtime overhead.

What does storing a weight in 4 bits mean? — 1 of 3

1. A large frozen base still occupies memory	LoRA reduces what is trained, but the original base weights remain needed for computation.

In plain English: Smaller memory requirements may enable a smaller GPU or a longer input; total cost savings are workload-dependent.

What does storing a weight in 4 bits mean? — 2 of 3

1. A large frozen base still occupies memory	LoRA reduces what is trained, but the original base weights remain needed for computation.
2. Four bits encode sixteen codes	Codes select approximate numerical levels, with scaling information. This compact representation saves base-weight storage.

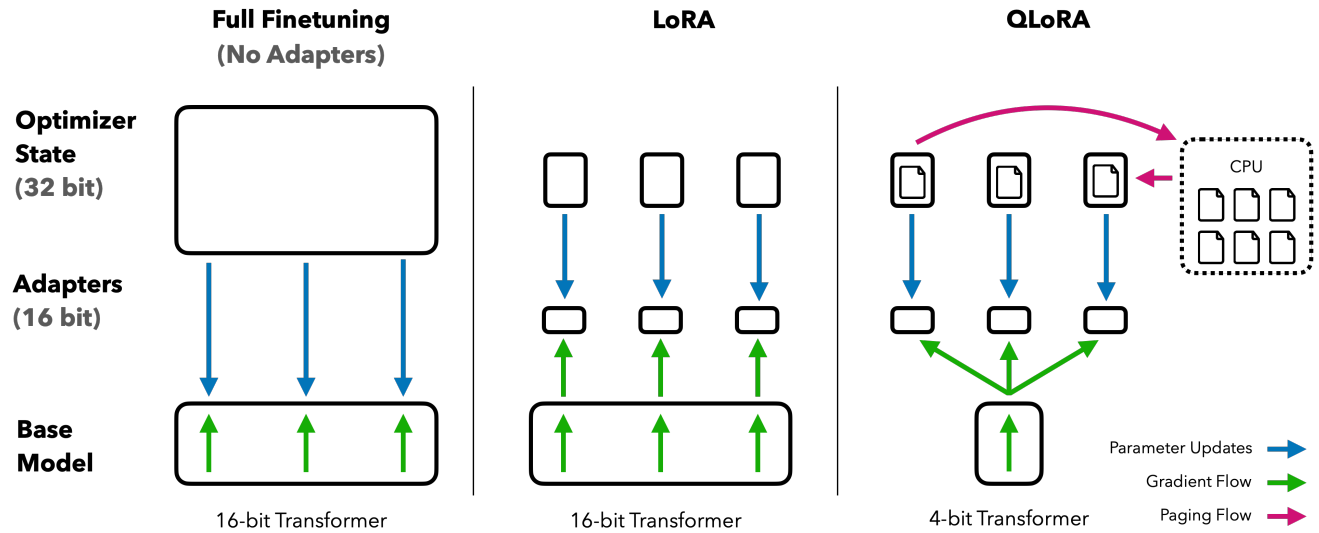
In plain English: Smaller memory requirements may enable a smaller GPU or a longer input; total cost savings are workload-dependent.

What does storing a weight in 4 bits mean? — 3 of 3

1. A large frozen base still occupies memory	LoRA reduces what is trained, but the original base weights remain needed for computation.
2. Four bits encode sixteen codes	Codes select approximate numerical levels, with scaling information. This compact representation saves base-weight storage.
3. Storage precision is not all computation	Adapters and calculations can use higher precision. Quantization adds approximation and does not guarantee the same quality or speed.

In plain English: Smaller memory requirements may enable a smaller GPU or a longer input; total cost savings are workload-dependent.

QLoRA: LoRA with a quantized base



LoRA: fewer trainable parameters. **QLoRA:** compact frozen base too.

Smaller memory requirements can enable a smaller GPU or fewer GPUs; speed and cost savings depend on the workload.

Check 2: can LoRA be used with SFT and DPO?

- A. No. LoRA replaces both training objectives.
- B. Only SFT; LoRA requires one target response.
- C. Yes. Either objective can train LoRA parameters.

Answer: C — objective and update method combine

Learning signal	Trainable parameters	Valid combination
SFT demonstrations	Full model or LoRA	SFT + LoRA.
DPO preference pairs	Full model or LoRA	DPO + LoRA.
Either, with quantized frozen base	LoRA adapters	QLoRA-style configuration.

SFT/DPO: what the loss rewards.
LoRA: how the parameter change is represented.

Why move beyond demonstrations?

- Writing one ideal answer can be difficult or expensive.
- A reviewer may find it easier to compare two existing answers.
- Preference data captures which response is better under a stated criterion.

Example: compare two summaries for factual support and clarity, rather than ask the reviewer to write a new summary.

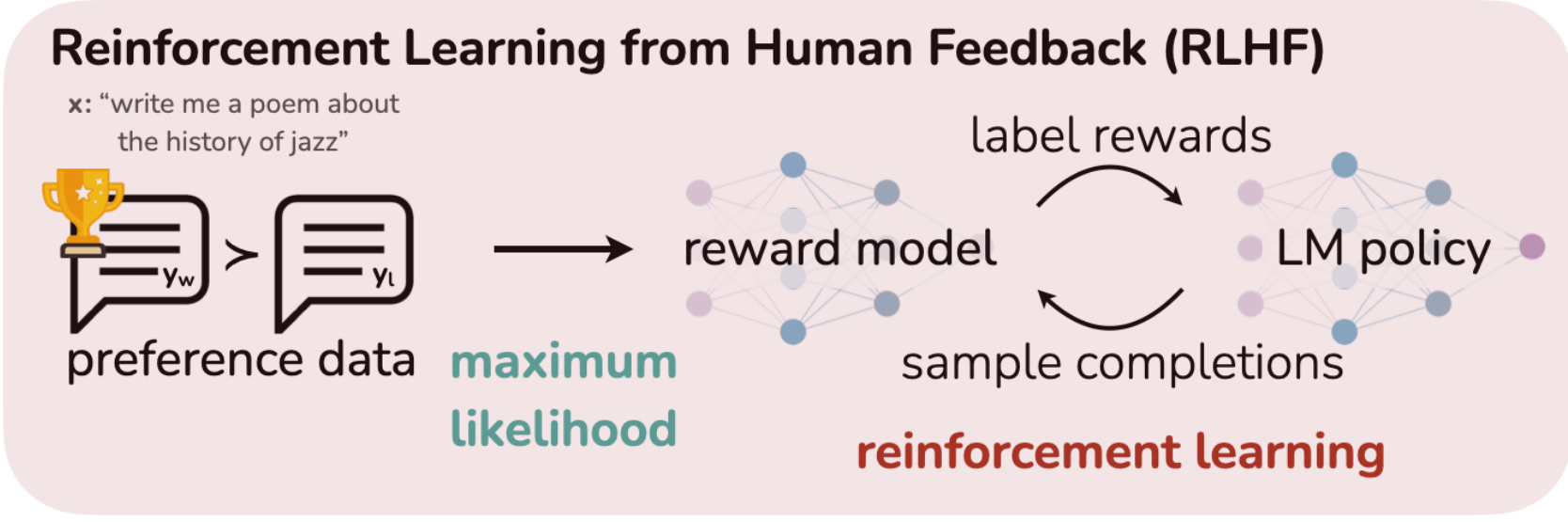
RLHF: the goal; PPO and DPO: different routes

Goal: use human feedback to improve responses.

Conventional reward-model + PPO route	DPO route
Learn a reward model, then optimize responses against its scores.	Train directly on recorded chosen/rejected response pairs.
Includes an online generation-and-scoring loop during policy training.	Standard offline DPO does not require that loop or a separately trained reward model.

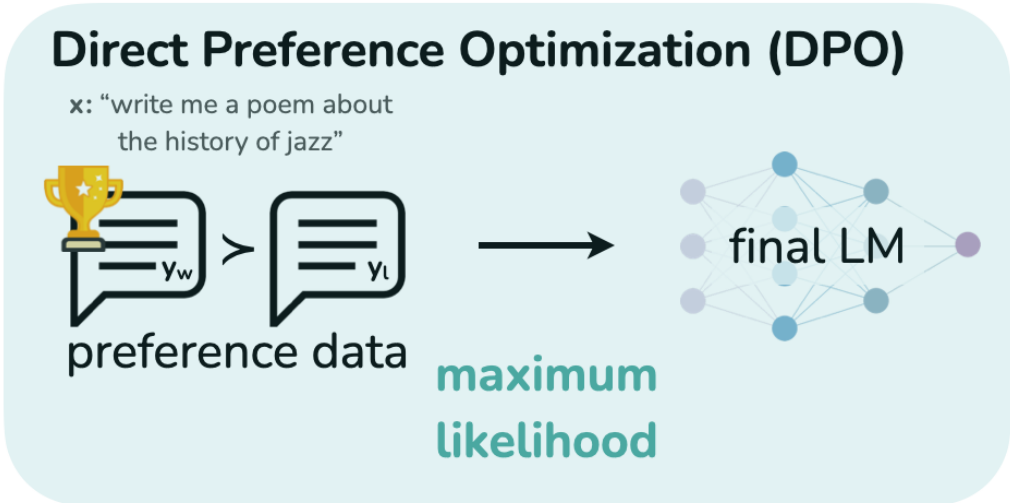
DPO is an alternative preference-training recipe, not the opposite of learning from human feedback.

RLHF: reinforcement learning from human feedback



Two jobs: learn a reward predictor; then use it to score generated responses and update the language model.

DPO removes the reward-model training loop



One training path: recorded preference pairs directly update the language model.

No separate reward model or on-policy generation loop in standard offline DPO.

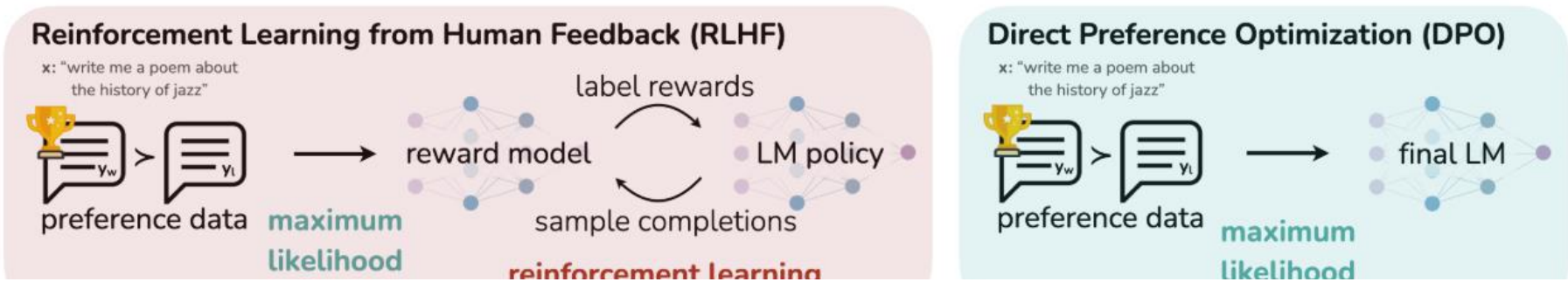
The original lecture makes the same comparison

Try to simplify PPO by..

- Getting rid of the reward model
- Getting rid of any on-policy stuff (rollouts, outer loops etc)

Instead

- Take gradient steps on log-loss of good stuff
- Take negative gradient steps on bad stuff (appropriately weighted).



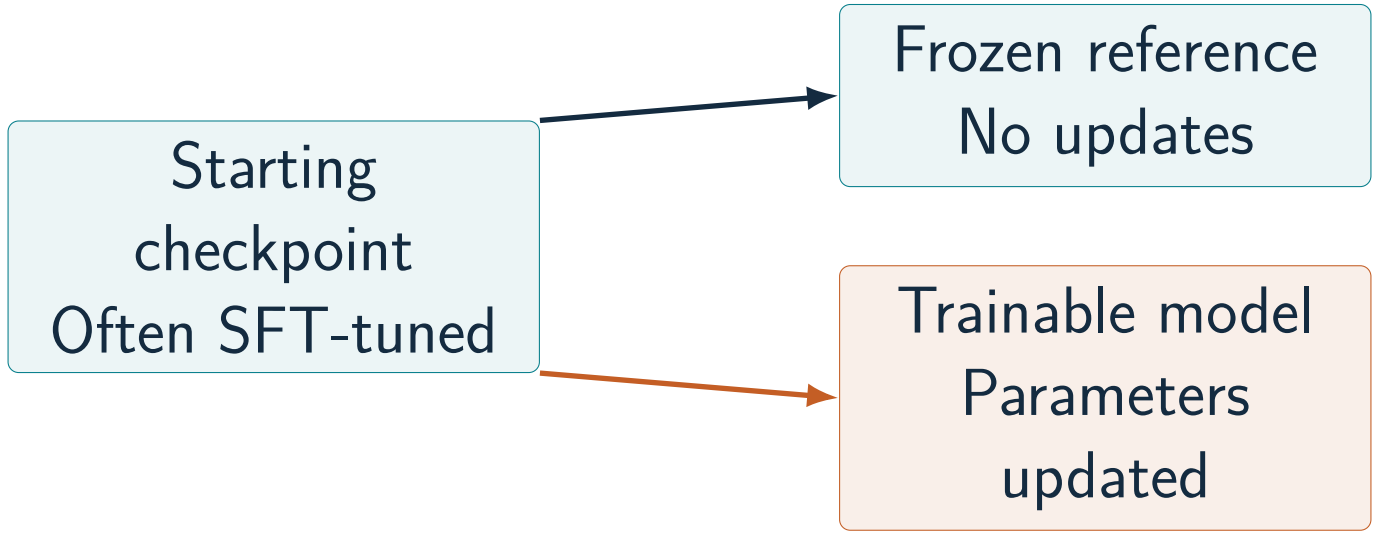
A DPO record: the preference label is explicit

Prompt and evidence	Chosen	Rejected
Meeting moved from Tuesday to Friday. Which day now?	Friday.	Tuesday.
No meeting day is provided. Which day?	The day is not specified.	Tuesday.

Stored fields: `prompt`, `chosen`, `rejected`.

The label is a preference, not an independently verified truth probability.

The reference model is a frozen starting point



Both score the same chosen and rejected responses.

Two meanings of frozen: DPO reference copy versus LoRA base weights.

The reference model is a frozen starting point — 1 of 3

1. Begin with a useful checkpoint	Often this is an already instruction-following model. A new local SFT stage is common but not always required.

In plain English: Freezing the DPO reference is a different role from freezing base weights while training LoRA adapters.

The reference model is a frozen starting point — 2 of 3

1. Begin with a useful checkpoint	Often this is an already instruction-following model. A new local SFT stage is common but not always required.
2. Keep one reference fixed	It scores the same candidate answers without updating. It is not the reviewer, dataset or source of factual truth.

In plain English: Freezing the DPO reference is a different role from freezing base weights while training LoRA adapters.

The reference model is a frozen starting point — 3 of 3

1. Begin with a useful checkpoint	Often this is an already instruction-following model. A new local SFT stage is common but not always required.
2. Keep one reference fixed	It scores the same candidate answers without updating. It is not the reviewer, dataset or source of factual truth.
3. Update the response model	DPO measures how the chosen-versus-rejected likelihood ratio changes relative to that reference.

In plain English: Freezing the DPO reference is a different role from freezing base weights while training LoRA adapters.

DPO rewards a relative preference shift — 1 of 3

1. Compare two answers to the same prompt	The dataset identifies a chosen and a rejected response. A preference criterion explains why one is favored.

In plain English: Illustration: moving a chosen/rejected ratio from 1/2 to 2 is a fourfold relative shift, not a probability that the answer is true.

DPO rewards a relative preference shift — 2 of 3

1. Compare two answers to the same prompt	The dataset identifies a chosen and a rejected response. A preference criterion explains why one is favored.
2. Compare current and starting preferences	Both models score both responses. The loss rewards improving the chosen/rejected ratio relative to the fixed reference.

In plain English: Illustration: moving a chosen/rejected ratio from 1/2 to 2 is a fourfold relative shift, not a probability that the answer is true.

DPO rewards a relative preference shift — 3 of 3

1. Compare two answers to the same prompt	The dataset identifies a chosen and a rejected response. A preference criterion explains why one is favored.
2. Compare current and starting preferences	Both models score both responses. The loss rewards improving the chosen/rejected ratio relative to the fixed reference.
3. Do not interpret preference as truth	A lower DPO loss means better fit to preference labels. It does not establish factual accuracy, calibrated beliefs or better business outcomes.

In plain English: Illustration: moving a chosen/rejected ratio from 1/2 to 2 is a fourfold relative shift, not a probability that the answer is true.

DPO: connect the explanation to the equation

$$\Delta = \log \frac{\pi_\theta(y^+ \mid x)}{\pi_\theta(y^- \mid x)} - \log \frac{\pi_{\text{ref}}(y^+ \mid x)}{\pi_{\text{ref}}(y^- \mid x)}, \qquad \mathcal{L}_{\text{DPO}} = -\log \sigma(\beta\Delta)$$

- x: prompt; y^+, y^-: chosen/rejected responses; π_θ: trainable response distribution with parameters θ; π_{ref}: frozen reference.
- Δ: change in log preference ratio; log: natural logarithm; $|$: conditional on; $\beta > 0$: scaling parameter.
- $\sigma(u) = 1/(1 + \exp(-u))$: sigmoid for input u; exp: exponential; $\mathcal{L}_{\text{DPO}}$: per-pair DPO loss, unitless.
- Read: reward a stronger chosen-versus-rejected ratio relative to the reference. Requires paired preferences, positive candidate probabilities and fixed scoring conventions; not a calibrated truth probability.

What assumptions support the DPO equation?

- A prompt x has two candidate responses and one recorded preference; ties are omitted in this binary setup.
- A scalar latent reward and a logistic pairwise model describe preferences.
- The reference stays fixed and assigns nonzero probability to the scored responses.
- The standard derivation links this preference model to a KL-regularized reward objective.

Real preferences can violate these assumptions. Training can still fit the labels without proving their validity.

Why use DPO after SFT? — 1 of 3

1. SFT establishes a response pattern	Reviewed demonstrations can teach instruction following, output structure and a specific interpretation skill.

In plain English: Common sequence: learn a useful response style first, then refine a defined preference. This is not mandatory for every model.

Why use DPO after SFT? — 2 of 3

1. SFT establishes a response pattern	Reviewed demonstrations can teach instruction following, output structure and a specific interpretation skill.
2. DPO adjusts recurring preferences	Comparisons can favor supported, useful responses over plausible alternatives without requiring a new ideal answer every time.

In plain English: Common sequence: learn a useful response style first, then refine a defined preference. This is not mandatory for every model.

Why use DPO after SFT? — 3 of 3

1. SFT establishes a response pattern	Reviewed demonstrations can teach instruction following, output structure and a specific interpretation skill.
2. DPO adjusts recurring preferences	Comparisons can favor supported, useful responses over plausible alternatives without requiring a new ideal answer every time.
3. Do not add a stage without a need	SFT may already be sufficient. Add preference training only when reliable labels address a measurable recurring failure.

In plain English: Common sequence: learn a useful response style first, then refine a defined preference. This is not mandatory for every model.

SFT, DPO and conventional RLHF: what differs?

Method	Training information	Core training path
SFT	Prompt + demonstrated response.	Predict demonstrated tokens.
DPO	Prompt + chosen/rejected pair.	Optimize relative preference using a reference.
Conventional RLHF	Comparisons plus generated responses.	Train reward model; optimize policy.

All depend on data quality and evaluation. LoRA can reduce trainable parameters.

Do training records have to be i.i.d.?

- LoRA is a parameterization: it has no independent-record requirement.
- SFT and DPO losses can be computed on dependent records.
- Usual generalization arguments often assume independent or suitably controlled samples from a relevant task distribution.
- Repeated documents and near duplicates reduce effective information; keep related cases together when splitting.

Example: 100 paraphrases of one document are not 100 independent source cases.

Preference improvement is not probability calibration — 1 of 3

1. Preferred means favored under a rubric	A reviewer may prefer clarity or brevity. That preference is not automatically a check of every factual statement.

In plain English: Calibration needs resolved labels and separate evaluation; SFT or DPO alone provides no such guarantee.

Preference improvement is not probability calibration — 2 of 3

1. Preferred means favored under a rubric	A reviewer may prefer clarity or brevity. That preference is not automatically a check of every factual statement.
2. Correct means supported and accurate	A clear answer can still be false. A correct categorical answer does not validate its confidence number.

In plain English: Calibration needs resolved labels and separate evaluation; SFT or DPO alone provides no such guarantee.

Preference improvement is not probability calibration — 3 of 3

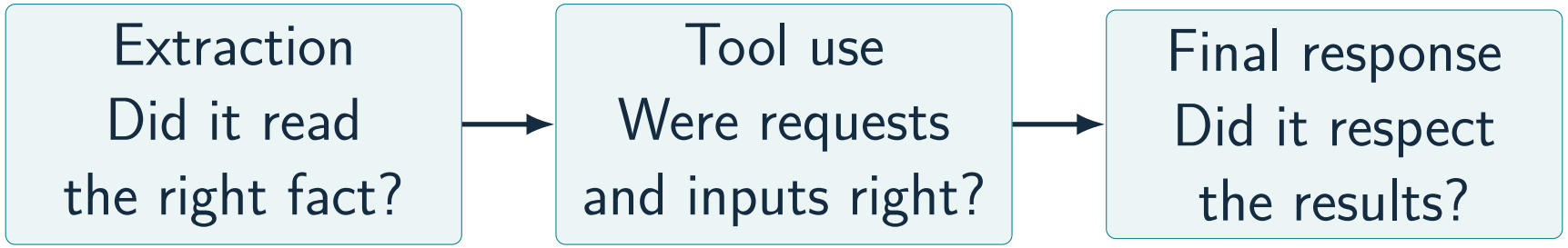
1. Preferred means favored under a rubric	A reviewer may prefer clarity or brevity. That preference is not automatically a check of every factual statement.
2. Correct means supported and accurate	A clear answer can still be false. A correct categorical answer does not validate its confidence number.
3. Calibrated means matching frequencies	For a defined event, cases scored near 80 percent should resolve positively about 80 percent of the time in the relevant population.

In plain English: Calibration needs resolved labels and separate evaluation; SFT or DPO alone provides no such guarantee.

Check 3: what would justify using the update?

- A. Lower loss on the training examples.
- B. More polished responses preferred by reviewers.
- C. Better held-out task performance with required checks preserved.

Answer: C — test behavior in the workflow



Compare base and adapted models on the same unseen cases.

A better average can conceal a new failure in a critical step.

Class exercise: identify what each setting changes

- Open `03_post_training.ipynb`; run setup C01.
- C02: change the masked prompt probability; inspect mean loss.
- C03: compare ranks, then trace input [2,1] to output [3,0].
- C04: compare chosen/rejected score pairs and DPO loss.
- These numerical examples explain the mechanism; Section 4 examines saved adaptation outcomes.

Exercise review: three distinct answers

- Loss mask: selected response tokens are scored; prompt context remains visible.
- Rank: doubling the adapter rank doubles its trainable parameter count for fixed target matrices.
- Preference: the reviewer specifies which response is better and why.

Choose the intervention that matches the problem

Observed problem	First intervention to investigate
Required information is absent.	Retrieval, source access and context construction.
Recurring task behavior remains wrong.	Instructions, demonstrations and SFT evaluation.
Responses violate a defined preference.	Reviewed comparisons; consider DPO.
An action exceeds delegated authority.	Enforced application permissions.

Training changes behavior; it does not replace evidence or authorization.

Objectives reviewed — next, run the experiment

Objective	What we have established
SFT	Demonstrations, teacher forcing and scored tokens.
LoRA / QLoRA	Trainable update, rank and base storage.
DPO / RLHF	Preference pairs, reference and reward pathways.
Evaluation	Independent cases and workflow checks.

Section 4: prepare data, run or inspect an adaptation, compare failures.

The question is whether the update improves the behavior we need.

Lunch

12:30–1:15

STAC AI BOOTCAMP NYC



Hands-on adaptation: from hypothesis to evidence

Matthew Dixon, PhD, FRM
BeliefLens

21 October 2026 | 1:15–2:15

matthew.dixon@belieflens.org

Start with the behaviour you want to change

Notebook: `04_adaptation_lab.ipynb`

Name a checkable failure.

"Make the model better" does not tell us what to train or how to judge success. Choose a recurring behaviour that a reviewer can recognize.

Concrete example

A request was waiting yesterday and approved today. The model keeps reporting the old status.

A useful target is narrow enough to check but broad enough to recur. We can then ask whether the same change helps on new examples.

A training run produces a candidate

Decide whether to use it.

We compare the candidate with the unchanged model on the same cases. Count both errors it fixes and errors it introduces.

Concrete example

Correctly reading the new approval is useful only if the change does not also invent approvals when no update exists.

A changed answer is not automatically a better answer. The comparison must tell us which requirements are now met and which have become less reliable.

Our route: six decisions in order

Decision	Why we need it
Diagnose	Establish which component failed.
Specify	Write the input and the desired response.
Protect	Reserve cases for selection and final assessment.
Configure	Choose the permitted update and resources.
Compare	Inspect matched outputs and regressions.
Qualify	Accept, revise or reject for a defined scope.

Each decision supplies what the next one needs. Choosing a training setting becomes meaningful only after we know the behaviour and comparison it must support.

What you will be able to explain

Ability	What you should be able to point to
Define the learning target	An input, a reviewed response and supervised tokens.
Explain the update	Frozen weights and trainable adapter matrices.
Protect the comparison	Distinct training, validation and final-test roles.
Compare behaviour	Corrections and regressions on identical cases.
Choose a repair	A failed handoff and its tested intervention.

The goal is to explain an experiment to another reviewer. Point to the input, update and observed outcomes rather than relying on the name of the training method.

First check whether the information arrived

Missing evidence calls for retrieval.

If the latest update never reached the model, there is no supplied update for it to interpret. Inspect the actual context before changing weights.

Concrete example

The document collection contains today's approval, but the prompt includes only yesterday's waiting status.

First confirm that the required information reaches the model. Then test whether it interprets that information correctly; this separates retrieval from interpretation.

Then locate the remaining failure

Interpretation and handoff errors need different tests.

Repeatedly selecting the old status from correct context can justify adaptation. Correct extraction followed by a corrupted tool argument calls for application repair.

Concrete example

Model output: approved. Database request: waiting. Repair the code that builds the request.

Several faults can produce the same final answer. Recording intermediate values helps us distinguish a model interpretation problem from a later software defect.

Write the input and target before training

Make the contract explicit.

Define what information the model receives, what response is acceptable and how missing or contradictory information should be represented.

Concrete example

Input: old status + latest update. Target: current status + supporting words. If no current update exists, return unknown.

The target should say what to do when support is absent, not only when the answer is obvious. Otherwise demonstrations may teach a confident response to incomplete information.

Define success before inspecting outcomes

Keep the baseline and assessment fixed.

Use the unchanged model with the same prompt and evidence. Evaluate unseen records and inspect difficult subsets, including missing updates and contradictory records.

Concrete example

A candidate that improves easy approvals but invents approval on missing-update cases may fail the intended task.

A fixed comparison prevents us from changing the definition of success to suit the result. Difficult cases reveal whether an apparent improvement is dependable.

One labelled example: what is input, what is target?

Role	Illustrative record
System instruction	Extract the latest supported status; otherwise return unknown.
User input	Old status: waiting. Latest update: the request was approved.
Reviewed assistant target	Status: approved. Evidence: the request was approved.

During training the reviewed assistant response supplies the tokens to learn. During use, only the instruction and evidence are supplied; the model must produce the response itself.

The target teaches the behaviour we reward

A wrong target teaches the wrong answer.

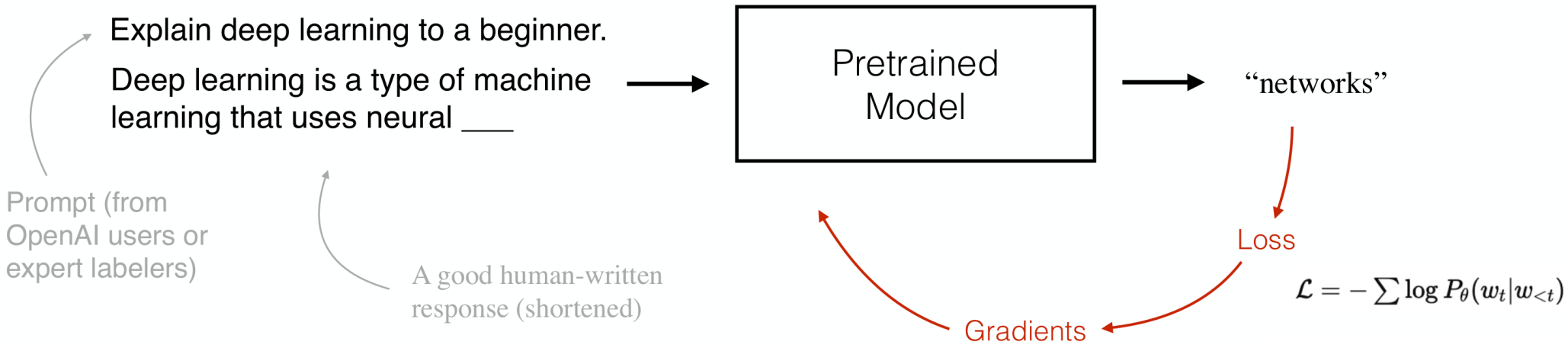
Training increases the probability of reviewed response tokens. Including a supporting quotation teaches a different output contract from training on a status word alone.

Concrete example

Input: old status waiting; no current update. ewline Reviewed target: Status: unknown. Evidence: no current update supplied.

Reviewing labels is part of engineering the behaviour. A consistent but incorrect target can be learned successfully while making the application less useful.

How a reviewed response changes the model



The model predicts a target token. A loss penalizes low probability for that token. Gradients guide a small change to trainable parameters.

The gradient describes how a small parameter change affects the prediction loss. Reducing that loss is a learning step; whether the resulting answers help still requires evaluation.

The loss scores the response tokens we choose

$$\mathcal{L}(\theta) = -\frac{1}{\sum_t m_t} \sum_t m_t \log p_\theta(y_t \mid x, y_{<t})$$

- θ: parameters; $\mathcal{L}$: mean loss; x: prompt; t: response token index; y_t: target token; $y_{<t}$: earlier target tokens.
- p_θ: model probability; m_t: 1 for scored tokens, 0 otherwise; Σ: sum; log: natural logarithm; $|$: conditional on.
- Read: average negative log probability over selected demonstrated tokens. Require at least one scored token; loss is nats per token.
- In this lab inspect the stored masking/training specification; low loss alone cannot qualify the full workflow.

Teacher forcing supplies the correct earlier words

$$p_\theta(y \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid x, y_{<t})$$

- x: prompt; y: full demonstrated response; T: response length; t: token position.
- y_t: current target token; $y_{<t}$: preceding demonstrated tokens; θ: model parameters; p_θ: probability.
- $\prod$: multiply across positions; $\mid$: conditional on. Probability is unitless.
- Read: sequence probability is the product of next-token probabilities. During training earlier target tokens are supplied; during generation earlier model outputs are used.

The loss mask does not hide the input

Positions	Visible as context?	Contribute to loss?
Instruction + user record	Yes	No: mask 0
Reviewed response + chosen end token	Yes	Yes: mask 1

These are illustrative positions, not actual tokenizer IDs. Input words still help predict the response even when their direct loss is zero.

A zero mask removes a position from the direct penalty, not from the conversation. The evidence can therefore guide the answer without being a text passage the model is trained to repeat.

How examples are weighted changes what is learned

Specify the averaging rule.

Averaging all supervised tokens gives long responses more influence. Averaging each example's mean gives examples equal influence. Repeating a record increases its weight.

Concrete example

Oversampling a rare class can balance training, but it does not create new independent information.

Weighting defines which mistakes matter most to the optimizer. Check that the chosen balance matches the task before interpreting a lower average loss.

Inspect the processed training record

The model sees tokens, not your original file.

Check role separators, the response positions selected for loss and the sequence after truncation. A correct source record can become an incomplete model input.

Concrete example

A long request history ends with “approved.” A length limit removes that final update. The remaining input no longer supports approval.

Inspect one ordinary record and one long or incomplete record after preprocessing. These small checks can reveal a systematic problem before it is repeated across the whole dataset.

Give each dataset one job

Fit, select, then assess.

Training adjusts parameters. Validation chooses settings or a checkpoint. The final test assesses the selected candidate after those choices are frozen.

Concrete example

Practice questions teach; a mock exam guides selection; a final exam checks the chosen preparation.

The separation is about how information is used, not which folder contains it. Once a result influences a model choice, it is part of development.

Separate rows may still share information

Match the split to the claim.

Copies, revised documents, repeated entities and overlapping outcome windows can cross a row split. A later period with familiar entities does not test new-entity generalization.

Concrete example

Training on yesterday's version of a record and testing on a near-identical revision may reward familiarity.

The split should support the specific claim we intend to make. Success on later records from familiar companies does not establish success on unfamiliar companies.

Question: which result should select the checkpoint?

Choice	Selection rule
A	Inspect final-test performance after every epoch.
B	Use the validation metric defined before the run.
C	Choose the candidate with the most impressive example.

A checkpoint is a saved model state from a stage of training. Consider how each choice uses information, then commit to an answer before continuing.

Answer: B, use validation for selection

Keep final assessment separate.

Validation supports choosing a candidate. Training loss diagnoses fit. The untouched final test assesses that chosen procedure, including its selection decisions.

Concrete example

Choose epoch two using validation, freeze it, then assess it on the final records.

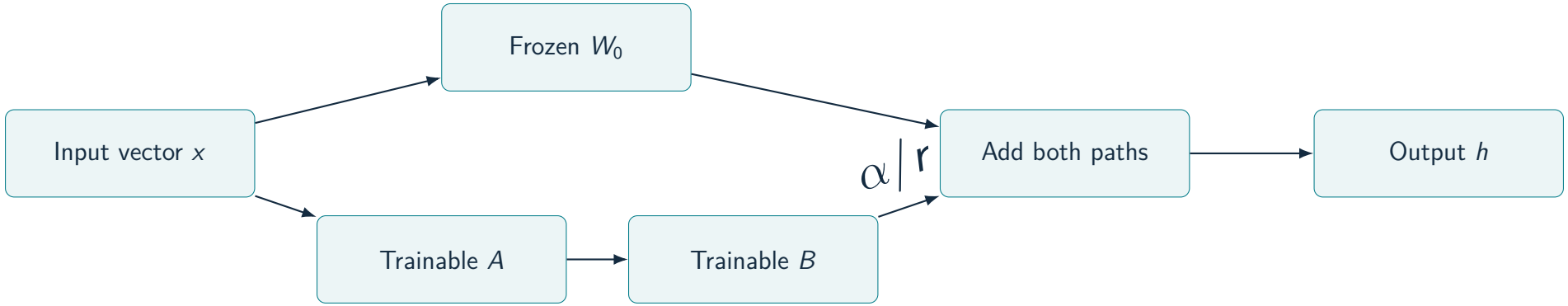
The selected candidate still has to pass the final assessment. If that assessment leads to another change, a new independent check is needed for the revised candidate.

LoRA learns a correction to a frozen transformation

$$h = W_0 x + \frac{\alpha}{r} BAx$$

- x: input vector; h: output vector; W_0: frozen base matrix, with output-by-input shape.
- A: trainable input-to-rank matrix; B: trainable rank-to-output matrix; r: positive integer rank; α: scale; α/r: unitless multiplier.
- Read: add a learned low-rank correction to the same base transformation. Vector units depend on the representation; weights map between them.
- Only the selected adapter matrices are updated. This is a parallel branch inside a layer, not a smaller replacement model or a layer inserted between every pair of layers.

The same input follows two paths



The upper path keeps the original transformation. The lower path learns a correction. Their outputs are added inside the model.

The correction is added inside a selected transformation, so it can change what later layers receive. It is not a separate smaller model replacing the original network.

x: input; h: output; W_0: frozen matrix; A, B: trainable correction matrices; r: positive rank; α: scale. The multiplier α/r is unitless.

Count the parameters that can change

$$A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}, \qquad N_{\text{adapter}} = r(d_{\text{in}} + d_{\text{out}})$$

- A, B: trainable matrices; r: rank; $d_{\text{in}}, d_{\text{out}}$: input/output widths; all dimensions are positive integers.
- $\mathbb{R}$: real-valued entries; $\in$: belongs to; $\times$: matrix row-by-column shape; N_{adapter}: number of trainable entries.
- Read: count entries in A plus entries in B. Counts are unitless; exclude biases and any other trainable modules.
- For 1024-by-1024 projection and rank 8: 16,384 adapter entries versus 1,048,576 base entries. Larger rank is a capacity choice, not guaranteed improvement.

LoRA makes an assumption we must test

A restricted update may be enough.

The product BA has rank at most r. We assume a useful task change can be approximated within that restricted family and in the selected transformations.

Concrete example

A compact correction may fix current-status extraction. It might also lack the capacity or placement needed for the task.

Low rank limits what the update can express. Whether that restriction is adequate is an empirical question for the task, not a guarantee supplied by the method.

BA: matrix product of trainable factors B and A; r: maximum correction rank, a positive integer.

Choose which transformations may change

Treat target modules as an experiment.

Query/value projections are one possible configuration. Adding key or other transformations changes the family of permitted updates and often the parameter budget.

Concrete example

Compare query/value against query/key/value using the same task and assessment, with a declared compute or parameter budget.

Changing the allowed transformations changes the experiment. Keep the assessment fixed so any benefit or regression can be compared under a clearly stated budget.

Choose the size and duration of the update

Settings control different things.

Rank controls update dimension; alpha its scale. Learning rate and steps control optimization. Sequence length and microbatch affect context and memory. Validation schedule controls selection evidence.

Concrete example

A longer sequence may retain the decisive exception, while a larger rank does not restore an exception that preprocessing removed.

Settings interact, so record the configuration instead of attributing a result to one knob afterward. Inspect actual trainable parameters and the context that reaches the model.

Memory-saving methods solve resource problems

Each saves something different.

QLoRA keeps a quantized frozen base and trains adapters. Gradient accumulation combines smaller batches before updating. Checkpointing recomputes activations to save their memory.

Concrete example

Accumulating eight small batches can replace one larger update batch. It cannot make a single oversized record fit.

First verify that a short run can complete an update, save and reload. Resource feasibility lets the experiment run; it does not show that the learned behaviour is better.

What one training step actually does

Operation	Meaning
`model(**batch).loss`	Predict target tokens and calculate their penalty.
`loss.backward()`	Calculate gradients for the permitted update.
`optimizer.step()`	Use gradients to change trainable parameters.
`optimizer.zero_grad()`	Clear gradients at the appropriate update boundary.

Only parameters permitted to train should be updated. Validation computes predictions and metrics without performing this optimizer step.

Saving and reloading is part of the experiment

A metric must identify a reproducible candidate.

Save the adapter, compatible base revision, tokenizer, template, configuration and evaluation outputs. Check that the saved candidate reloads before relying on it.

Concrete example

Two adapters with the same file name but different base revisions are not necessarily interchangeable.

A result is useful only if we know which saved state produced it. Preserve the associated data, template and configuration so another reviewer can repeat the comparison.

Choose the route that answers your question

Route	What actually happens
Recorded adaptation	Inspect real saved training and paired outputs; no key or GPU.
Controlled workflow repair	Change a supplied faulty value and recompute dependencies.
Optional instructor training	Perform fresh parameter updates on prepared hardware.

The default activities run without a provider key. Recorded results support inspection, while the controlled repair lets you change a value and observe its consequences.

Exercise A: inspect what was trained

Open the readable adaptation notebook.

Run sections 1–2 of `04_actual_adaptation_readable.ipynb`. Find the input, target, trainable modules and selected epoch.

Concrete example

Be able to finish: "The model received [input] and was trained to output [target]."

Look for the exact target rather than assuming it from the task name. Confirm whether the model learned a class token, a prose answer or another response format.

Exercise A: compare one matched case

Change the case, then reveal the outcome.

In section 3 change `CASE_INDEX`. Predict before revealing the reference outcome. Run sections 4–6 to inspect checks, aggregate results and split dates.

Concrete example

State whether the adapter's changed answer corrected an error or introduced one.

One interesting case explains a change; the full set assesses its pattern. Keep corrections and regressions in the comparison instead of choosing only favourable examples.

The recorded run: what these results refer to

Recorded item	Value
Base	Qwen2.5-0.5B-Instruct; 494,032,768 base parameters.
Update	Query/value LoRA; rank 8; alpha 16.
Training data	527 distinct windows; 777 balanced records per epoch.
Resources	One T4; recorded duration 28.6 minutes.
Selection	Epoch 2 by validation loss.

The balanced records include repeated influence from existing windows. The model size and runtime identify this run; they are not a promise about another dataset or environment.

A real labelled record, condensed for readability

Field	Verified saved-record excerpt
Input cutoff	CVX; 9 October 2025, 06:30 UTC.
Price input	21 daily closes through 8 October; latest USD 153.7400; prior 20-session return -2.5667%.
Other input	News, financial facts and SPY-relative context.
Target output	`C`: realized five-session excess return above +1%.

The future return becomes a target only after the outcome is observed. It must not be supplied as part of the earlier forecasting evidence.

The financial target is not a truth certificate

The model learns a class-token mapping.

The target does not certify evidence quality, predict a profitable authorized trade or provide calibrated confidence. Inspect the exact historical prompt contract.

Concrete example

The saved system requests A/B/C, while its nested question requests named signals and abstention. That mismatch is an audit finding.

A timestamp check can pass even when the news is irrelevant to the company. Evidence eligibility and target prediction are different requirements and should be inspected separately.

Why epoch two was selected

Loss	Epoch 1	Epoch 2	Epoch 3
Mean logged training	0.3783	0.3563	0.3510
Validation	0.4040	**0.3692**	0.3770

Training fit keeps improving. The separate validation metric is lowest at epoch two. That identifies the candidate to assess next.

Validation selects which saved state to assess. The test of complete predictions still follows; a lower token loss alone does not establish a better financial decision.

Count corrections and regressions together

Same 16 cases	Base	Adapter
Correct labels	6/16	3/16
Base errors corrected	—	3
Previously correct cases broken	—	6

The adapter changes behaviour, but this comparison does not establish improved overall label accuracy.

Six previously correct cases were lost and three previously wrong cases were fixed. That paired account shows why changed behaviour alone is not an improvement claim.

Why a different metric can appear to improve

Balanced accuracy gives classes equal weight.

It rises from 33.3% to 40.7%, while ordinary accuracy falls. Here there are six negative outcomes, one neutral and nine positive. The single neutral case has large influence.

Concrete example

Correctly recognizing that one neutral outcome contributes one third of the class-averaged metric.

Metric choice determines how errors are weighted. Report the class counts and regressions so a single rare outcome cannot make the result look more convincing than it is.

The final answer does not locate the failure

Inspect the handoffs. Follow evidence → extraction → tool arguments → calculation → allowed action. Check the recorded input and output at each step.

Concrete example

The source and extraction say approved, but the application's request says waiting. That mismatch is observable.

The graph records observable relationships between artifacts. It can locate a failed check without claiming to reveal the model's hidden internal reasoning.

Use the graph to target a repair

Recheck what depends on the changed value.

If an input is repaired, reassess its dependent calculations and claims. Keep independent permissions and other faults visible.

Concrete example

A corrected extraction can repair a report while a separate wrong calculator argument still blocks the action.

A changed input may affect several dependent outputs, but it need not change all of them. Verify the actual recomputation and retain any separate unresolved requirement.

Exercise B: inspect the actual calculator request

Compare source values with tool arguments.

Open `04_workflow_diagnosis.ipynb`. Set `CASE_ID='arguments'` in C02. The source says consensus 1.00, but the request passes 2.00.

Concrete example

Actual earnings: 1.20. Correct expected earnings: 1.00. The input discrepancy appears before the calculator runs.

The calculator can only use the values it receives. Compare those values with the evidence before blaming its formula or retraining the language model.

Correct arithmetic can use the wrong input

$$\text{surprise} = \frac{\text{actual} - \text{consensus}}{|\text{consensus}|}$$

Inputs sent to calculator	Result
Correct: actual 1.20; consensus 1.00	$(1.20 - 1.00)/1.00 = +20\%$
Faulty: actual 1.20; consensus 2.00	$(1.20 - 2.00)/2.00 = -40\%$

Both calculations are arithmetically correct. Only one uses the source-supported consensus.

The inputs must share company, period, units and basis, and consensus must be nonzero. Correcting arithmetic cannot compensate for using an unsupported input.

Actual and consensus are earnings per share in the same currency and period. Bars mean absolute value; require nonzero consensus. Surprise is unitless, shown as a percentage.

Exercise B: repair one fault, then inspect what remains

Predict before applying the repair.

Set `REPAIR_ID='tool_request'` in C04. Then try `two_faults`: repair extraction alone and inspect the remaining argument defect.

Concrete example

A repaired figure can restore one report while the trading path remains withheld.

A successful local repair is not permission to mark the whole workflow correct. The second case demonstrates why remaining checks and independent authority still matter.

Question: which intervention addresses this defect?

The model extracts the correct value. Application code changes it before calling the calculator.

Choice	Next step
A	Train on more examples of the desired final action.
B	Repair the argument handoff and rerun its checks.
C	Increase adapter rank until the action changes.

Use the recorded mismatch to choose an intervention. Before continuing, state which observation supports your choice and what you would expect to change afterward.

Answer: B, repair the argument handoff

Verify the input-to-source relationship.

Correct the application request, then recompute affected outputs. Retain unrelated faults and independent permission checks.

Concrete example

An approved request may now be recorded correctly, while the operator still lacks authority to release it.

The repaired request should agree with its supporting evidence. A newly correct final answer would not by itself prove that the handoff was repaired.

Decide using the full experimental evidence

Accept, revise or reject for a defined scope.

Review task performance, uncertainty, regression cases, evidence checks and execution permissions. A falling loss curve is only one part of that record.

Concrete example

A model that improves ordinary cases but invents unsupported approvals may need rejection or a narrower permitted use.

A narrow, justified use may be acceptable even when a broad claim is not. State the remaining uncertainty and the consequences of the errors that remain.

Make the decision reproducible and accountable

Keep the artifacts and name the owner.

Record data and split versions, model and adapter revisions, configuration, outputs and limitations. Specify permitted scope, monitoring and rollback.

Concrete example

Another reviewer should be able to identify the selected candidate and explain why it was accepted or rejected.

Reproducible evidence makes a decision inspectable, but it cannot cure a biased assessment. The owner must also specify when new evidence triggers review or rollback.

Explain your conclusion in three sentences

1. What observable behaviour or handoff failed?
2. Which intervention would you test, and why?
3. What evidence would make you accept or reject it?

Use the input, target, permitted update, matched results and workflow checks to support your answer.

A useful conclusion may be that no model training is needed. The important result is a justified intervention and a clear test of whether it worked.

Section 4: objectives revisited

Objective	What you should now be able to explain
Diagnosis	Locate the first mismatch in source, extraction or tool arguments.
Intervention	Predict and check what changes after one repair.
Training decision	State a specific behaviour SFT should improve.
Qualification	Compare the base and adapter on matched, independent cases.

STAC AI BOOTCAMP NYC



From language to credible predictions and decisions

Matthew Dixon, PhD, FRM
BeliefLens

21 October 2026 | Section 5 · 2:15–3:00

matthew.dixon@belieflens.org

Why this section: can the result survive deployment?

Notebook: `05_alpha_risk.ipynb`

Decision: whether an apparent improvement merits a forward trial.

- Section 4: does adaptation improve the task on held-out cases?
- Section 5: could this workflow have made these decisions at the time?
- A fluent explanation, a correct extraction and a profitable action are different achievements.

Roadmap: from information to a credible decision

Each stage resolves a different reason a backtest can mislead.

Stage and time	Question answered
1. Define the task · 7 min	What is predicted, when, and for which action?
2. Reconstruct the past · 10 min	Which evidence, labels and model were available?
3. Test the complete workflow · 10 min	Does it beat a fair baseline after costs?
4. Diagnose and replay · 13 min	What caused the result, and can we reproduce it?
5. Decide what to claim · 5 min	What remains unproven before a forward trial?

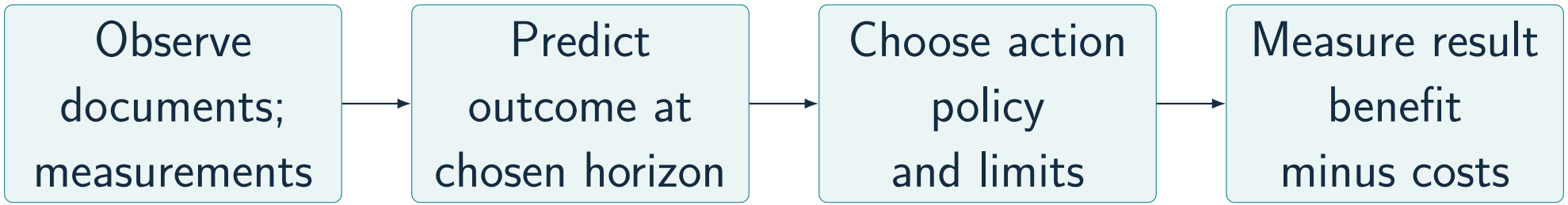
Five outcomes for this session

Use these as the acceptance checklist for a proposed experiment.

- Specify the target, horizon, information boundary and action.
- Construct a chronological replay with valid source and label availability.
- Compare against baselines without tuning on the final test.
- Separate predictive quality from decision value and costs.
- Locate a failure in evidence, interpretation, policy or execution.

Separate observation, prediction and action

Decision: which component is responsible for the claimed improvement.



A language model may extract evidence, produce a feature, or propose an action. Specify which role is being tested.

Define the prediction before choosing the model

$$\widehat{y}_{t,h} = f_{\theta_t}(x_t), \qquad y_{t,h} = \text{outcome over } (t, t+h].$$

- t: decision time; h: forecast horizon, in the same time unit.
- x_t: inputs available at t; f: prediction function; θ_t: its fitted parameters available then.
- $\widehat{y}_{t,h}$: predicted outcome; $y_{t,h}$: observed outcome. A hat marks an estimate; $(t, t+h]$ excludes the start and includes the end.
- Example: use reports available at 09:00 to predict failure over seven days. Define the outcome and intervention costs first.

A point-in-time labelled training record

Field	Illustrative record
Decision	Fictional stock ABC, Monday 09:00.
Input available then	Archived release and market data available by 09:00.
Outcome label	1 if the next five-trading-day excess return is positive; 0 otherwise.
Label availability	Only after the outcome window and any data delay.

The future outcome is a training label, never part of the original forecast input.

Changing regimes: what must remain useful?

- A relation learned in one period may fail when liquidity, behavior or market structure changes.
- An LLM can propose news-based indicators or hypotheses; their usefulness still needs testing.
- Evaluate later periods and stressed regimes; define triggers for reassessment.

No strict stationarity is needed to write a forecast equation. Generalization still needs some stability or an effective, validated adaptation rule.

Turn language into a reproducible measurement

Decision: what exactly will be tested across time.

Component	Record it explicitly
Input	Source IDs, saved text, availability times and versions.
Transformation	Prompt, model version, retrieval settings and parser.
Feature or forecast	A defined quantity: delay duration, exposure, event probability.
Failure handling	Missing, contradictory or unparseable evidence leads to a declared fallback.

Chronological rows do not guarantee historical knowledge

$$\mathcal{I}_t = \{\text{records and versions actually available by } t\}, \qquad x_t = g(\mathcal{I}_t).$$

- t: decision time; $\mathcal{I}_t$: information set available then; braces denote a set.
- g: declared input-building procedure; x_t: resulting model input.
- Event time, availability time and revision time can differ. All clocks must share a specified time zone.
- Read: construct the input only from eligible historical versions. A date-sorted table alone does not prove eligibility.

A saved URL is not a saved historical document

Decision: which version may enter a historical replay.

Time	What the workflow could see
Monday 09:00	Report version 1: demand grew 2%.
Monday 09:05	Decision made using version 1.
Tuesday 12:00	Same URL updated: growth revised to 5%.
Replay today	Fetching the URL now returns information unavailable on Monday.

Keep: content snapshots, version identifiers, availability records and extraction history.

The model has a knowledge history too

Decision: retrospective analysis or a feasible historical strategy?

- Filtering retrieved documents does not remove knowledge encoded in model weights.
- A current model applied to an old event may recognize its later outcome.
- Use documented model vintages where possible; test prospectively on newly resolved events.
- If historical availability cannot be established, narrow the claim to retrospective extraction.

Question: which claim does the evidence justify?

Test objective: distinguish source timing from model timing.

- A 2026 model reads only documents dated before each 2018 decision.
- Its simulated strategy performs well during 2018.
- A. This proves the model could have traded profitably in 2018.
- B. This supports a retrospective result, with model-knowledge contamination unresolved.
- C. The document filter guarantees that no later information was used.

Answer: B — the model vintage remains unresolved

Decision: what further evidence is required before claiming foresight.

- The document filter restricts retrieval, not the contents of the weights.
- A reproducible historical extraction result can still be informative.
- A forward trial with frozen settings tests prediction under a defensible information boundary.

Evaluate as the forecast origin moves forward

Decision: whether the model is learning only from the past.



Freeze each prediction before observing its outcome. Evaluate the horizon actually required.

A past row may still have an unresolved label

$$\mathcal{D}_t = \{(x_i, y_i) : a(x_i) \le t,\ a(y_i) \le t\}.$$

- t: training cutoff; i: example index; x_i: input; y_i: resolved outcome label.
- $a(z)$: availability time of record z; $\mathcal{D}_t$: eligible training pairs. Colon means "such that; $\le$ means no later than.
- Read: both the input and its label must be available before fitting. Use one time zone and include reporting delays.
- A Monday input with a seven-day outcome is not a resolved example for Friday training.

Implement the split; do not mistake it for an audit

Decision: what the library handles and what you must supply.

Python building block

```
TimeSeriesSplit(n_splits=3, gap=5)
```

- Produces ordered train/test indices; a gap excludes rows near the boundary.
- Choose the gap from label overlap and availability, not an arbitrary convention.
- The splitter cannot authenticate document versions, model vintages or timestamps.
- For irregular events, use actual availability times and outcome intervals.

Fit every learned transformation inside the training window

Decision: where preprocessing and calibration belong.

Leakage route	Repair
Normalize using all observations	Estimate the scaler from the training window only.
Select features using final test outcomes	Select within training/validation; leave final test untouched.
Tune a prompt after inspecting test failures	Treat those cases as development; reserve another test.
Fit a probability mapping on test labels	Fit on separate available calibration data.

A convincing comparison holds the opportunity set fixed

Decision: whether the language model adds incremental value.

Comparator	What it establishes
No action / existing operating policy	Whether changing behavior is worthwhile at all.
Simple feature or deterministic rule	Whether complex language processing adds useful information.
Same model, different evidence	How evidence quality changes results.
Same evidence, different policy	How much is due to the action rule.

Searching many designs changes what the best score means

Decision: how much confidence to place in a selected winner.

- Prompts, models, horizons, filters and thresholds are all research choices.
- Selecting the best historical result can select noise.
- Log attempted designs; separate tuning periods from final evaluation.
- Repeat across later periods and examine the whole search process.

Language likelihood and event probability answer different questions

$$P(\text{answer tokens} \mid \text{prompt}) \neq P(Y_{t,h} = 1 \mid \mathcal{I}_t).$$

- P: probability; $\mid$: conditional on; $\neq$: not generally equal.
- t: decision time; h: horizon; $Y_{t,h}$: binary event indicator ($1 =$ event, $0 =$ no event).
- $\mathcal{I}_t$: information available at t. Both probabilities are unitless, but describe different objects.
- Token likelihood concerns wording. Event probability requires a defined outcome and validation against resolved cases.

Evaluate probabilities and consequences separately

$$\text{Brier} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2, \qquad y_i \in \{0, 1\}.$$

- N: number of evaluated cases; i: case index; Σ: sum over those cases.
- p_i: predicted event probability; y_i: observed binary outcome. $\in$ means belongs to; 1 = event and 0 = no event.
- Read: average squared probability error. Score and probabilities are unitless; lower is better for this loss.
- Use a common resolved target and paired cases. The score measures forecast quality, not intervention costs or trading value.

Choose a policy using explicit consequences

$$a^*(x) = \arg\min_{a \in \mathcal{A}} \mathbb{E}[L(a, Y) \mid x].$$

- x: available input; Y: uncertain outcome; a: action; $\mathcal{A}$: permitted action set.
- $L(a, Y)$: loss in declared cost units; $\mathbb{E}[\cdot \mid x]$: conditional average; arg min: action attaining the smallest value; a^*: chosen action.
- Inspection example: C = inspection cost; D = failure loss; p = event probability. If inspection prevents all loss, inspect when $pD > C$.
- Equivalently $p > C/D$ for $D > 0$. Costs share units; p and C/D are unitless. Validate probability and intervention assumptions.

For trading, measure the return after implementation

$$R_{t+1}^{\text{net}} = w_t r_{t+1} - c\,|w_t - w_{t-1}|.$$

- t: rebalance time; $t+1$: next return interval; $t-1$: previous rebalance.
- w_t, w_{t-1}: current and previous portfolio weights; r_{t+1}: asset return; R_{t+1}^{net}: net portfolio return (decimal units).
- c: proportional cost per unit weight traded; $|\cdot|$: absolute value. The weight change measures turnover.
- Read: exposure times return, less trading cost. This simplified model excludes funding, market impact and variable slippage unless added explicitly.

For risk, count severity and timing as well as errors

Decision: whether warnings improve risk management.

Measure	Why it changes the conclusion
Breach or event frequency	Too few warnings may miss problems; too many may be costly.
Loss severity	One large miss can dominate many small correct decisions.
Clustering and regime	Concentrated failures may expose an unusable conditional forecast.
Lead time and response cost	A correct warning may arrive too late or trigger wasteful intervention.

Question: which improvement is decision-relevant?

Test objective: distinguish prediction accuracy from policy value.

- Model A catches 8 of 10 failures and triggers 90 unnecessary inspections.
- Model B catches 7 of 10 and triggers 10 unnecessary inspections.
- A. A must be better because recall is higher.
- B. B must be better because it raises fewer false alarms.
- C. We need the costs, severity and effectiveness of intervention.

Answer: C — specify the loss and intervention effect

Decision: compare policies on the same outcomes and assumptions.

- Count what happens after each permitted action, including no action.
- Include false alarms, missed events, delays and intervention costs.
- Report predictive metrics separately so policy choices remain visible.

Diagnose the chain before changing the model

Decision: which intervention would address the observed failure.

Stage	Test that isolates the problem
Evidence	Replace a stale or missing source with a verified eligible source.
Interpretation	Hold evidence fixed; compare extracted facts with reviewed labels.
Policy	Hold the signal fixed; recompute the permitted action.
Execution	Compare authorized intent with actual dispatched action and cost.

Class Exercise: did the signal predict or read the future?

- Open `05_alpha_risk.ipynb`; run setup C01, then C02.
- Compare valid information with future-derived signals, gate off and on.
- C03: inspect signal availability against decision time for each row.
- C04: change fees; compare net returns with the zero-return no-trade baseline.

What to inspect before accepting the attractive result

Decision: whether the result supports prediction, enforcement or neither.

Look at	Interpretation
Signal availability versus decision time	Late information makes that simulated prediction infeasible.
Blocked order and its reason	The gate enforced the declared information rule.
Gross versus net result	Costs may remove the apparent economic benefit.
Same opportunity and return ledger	Differences can be attributed to the changed setting.

From a promising replay to a forward trial

Decision: what must be frozen before the next outcome arrives.

- Freeze the target, eligibility rules, model, prompt, policy and cost assumptions.
- Timestamp predictions and proposed actions before outcomes resolve.
- Keep abstentions, missing inputs and rejected actions in the evaluation.
- Predefine review triggers; examine recent performance and failure groups.

Return to the five learning objectives

What you should now be able to explain.

Objective	Evidence from this session
Define a testable target	Write unit, decision time, horizon and outcome rule.
Respect the information boundary	Check source versions, model vintage and label availability.
Design a fair comparison	Use rolling origins, frozen choices and explicit baselines.
Measure decision value	Separate probability quality, costs and consequences.
Diagnose a failed workflow	Trace evidence, interpretation, policy and execution.

Reading map: use each source for a specific decision

Methods are attributed; workshop examples are explicitly illustrative.

Source	Use in this lecture
Hyndman & Athanasopoulos, FPP3 ch. 5	Forecasting workflow, rolling origins, baselines and quantile evaluation.
scikit-learn official guidance	Temporal splitting, leakage prevention and calibration.
FRED/ALFRED real-time periods	Observation dates versus information vintages.
Bailey et al., backtest overfitting	Why selecting a winner from many historical trials needs scrutiny.

Section 5: objectives revisited

Objective	What you should now be able to explain
Research	State a mechanism, horizon, counterevidence and refutation test.
Beliefs	Define a state or event and how its probability is evaluated.
Changing regimes	Explain why past calibration need not transfer.
Backtesting	Check document vintages, model history and label availability.

Break

3:00–3:15

STAC AI BOOTCAMP NYC



Evaluation and evidence attribution

Matthew Dixon, PhD, FRM
BeliefLens

21 October 2026 | 3:15–4:15

matthew.dixon@belieflens.org

The decision this lecture supports

Notebook: `06_evaluation.ipynb`

Is this system ready for this task—and what should we fix first?

- A benchmark score estimates performance on its tested cases.
- A diagnostic test isolates a specific capability or failure.
- An evidence trace tells us which sources and operations support a claim.

Roadmap: from a claim to a defensible test

Stage	Question	Time
1. Define	What exactly must work?	10 min
2. Measure	Which observations establish success?	10 min
3. Intervene	What changes should matter?	15 min
4. Diagnose	Which dependency explains the failure?	15 min
5. Decide	What can we claim and release?	10 min

Five learning objectives

- Write a task contract with observable success and abstention conditions.
- Choose separate measures of correctness, support, stability and calibration.
- Design paired interventions with repeat controls and uncertainty estimates.
- Distinguish evidence dependencies from causal effects inside a model.
- Turn failures into a repair, retest and bounded release claim.

Start with an evaluation contract

Specify before testing	Why it is needed
Input and allowed evidence	Defines what the system may know.
Output and permitted abstention	Makes success observable, including missing information.
Target cases and exclusions	Defines where the result applies.
Errors, costs and release tolerance	Determines which failures matter operationally.

A benchmark is a sample of the task

- Cover ordinary cases, difficult cases and cases that cannot be answered.
- Include shortcuts that work on easy cases but fail on carefully matched cases.
- Keep independent test families separate from development examples.
- Report results by failure type as well as overall.

A labelled evaluation record

Field	Illustrative record
Case and family	E01; meeting-update family.
Input	The meeting moved from Tuesday to Friday. Which day now?
Reviewed reference	Friday.
Support label	Friday is supported; Tuesday is superseded.
Missing-source variant	Expected behavior: withhold the unsupported answer.

Keep case families separate from development. Do not train on the final evaluation fixtures.

Correctness, support and stability are different

Property	Question	Possible failure
Correctness	Does it match the reference?	Correct answer by lucky guessing.
Support	Do eligible sources justify it?	Plausible claim with irrelevant citation.
Stability	Does irrelevant variation change it?	Same facts, different answer order.
Calibration	Does stated probability match frequency?	Confident errors on unseen cases.

Choose the measurement for the claim

Target	Observable	What it does not establish
Exact extraction / arithmetic	Exact match; recomputed value	Whether the source is eligible.
Free-text support	Atomic claim + cited span + rubric	Whether the model relied on it.
Operational behavior	Executed tool and action trace	Whether the answer is true.
Language similarity	Overlap or embedding similarity	Entailment, factuality or authorization.

Evaluate the evaluator

- Write a rubric with correct, unsupported, contradicted and unanswerable examples.
- Have independent reviewers label a shared sample; inspect disagreement.
- If a model assists grading, validate it against reviewed labels and retain reasons.
- Use accountable checks and review for release decisions.

Question: a correct answer cites the right document

What have we established?

- A. The model must have used that document.
- B. Correctness and a citation are observed; dependence still needs testing.
- C. The answer will remain correct after the document changes.

Answer: B—test dependence separately

- Check that the cited passage actually supports the claim.
- Change or remove that evidence under a defined intervention.
- Compare the outcome with a matched control.
- Interpret the result only for the tested intervention.

Three kinds of behavioral test

Test	What should happen?	Simple example
Minimum functionality	A basic capability works.	Extract the stated date.
Invariance	Meaning-preserving edits preserve the result.	Change the order of independent records.
Directional expectation	Relevant changes move the result appropriately.	Replace positive evidence with a contradiction.

Control one difference at a time

Arm	Held fixed	Changed
Baseline repeats	Evidence, wording, order, settings	Execution only.
Presentation intervention	Evidence content and task	Framing or record order.
Evidence intervention	Task and presentation template	One relevant fact or missing source.
Negative control	Relevant evidence and task	An unrelated passage.

Measure changes, not just final labels

$$\text{flip} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{a_i \neq a_i'\}, \qquad \text{TV}(q_i, q_i') = \tfrac{1}{2}\sum_a |q_i(a) - q_i'(a)|.$$

- n: paired cases; i: case index; a_i, a_i': selected actions before/after the intervention; prime marks the changed condition.
- $\mathbf{1}\{\cdot\}$: 1 if true, 0 otherwise; $\neq$: differs from; Σ: sum; $|\cdot|$: absolute value.
- $q_i(a), q_i'(a)$: normalized probabilities of action a in the same action set; TV = total variation.
- Both measures range from 0 to 1, unitless. Zero flips need not mean zero probability movement; correctness must also be checked.

Candidate scoring requires a fixed boundary

$$\ell(a \mid x) = \sum_{t=1}^{T_a} \log p(a_t \mid x, a_{<t}), \qquad q(a \mid x) = \frac{\exp \ell(a \mid x)}{\Sigma_{b \in \mathcal{A}} \exp \ell(b \mid x)}.$$

- x: prompt plus answer prefix; a: candidate token string; t: token position; T_a: token count; a_t: next token; $a_{<t}$: earlier candidate tokens.
- p: model token probability; ℓ: summed natural log probability; log: natural logarithm; exp: its inverse.
- $\mathcal{A}$: candidate set; b: alternative in that set; q: normalized candidate score; $\mid$: conditional on; Σ: sum.
- All scores are unitless. Fix tokenization and termination for every candidate. These are not calibrated event probabilities; absent API alternatives are unknown.

Calibration needs outcomes and a separate split

$$\Pr(Y = 1 \mid \widehat{p} \approx 0.8) \approx 0.8$$

- Y: observed binary correctness indicator ($1 =$ correct, $0 =$ incorrect).
- $\widehat{p}$: predicted probability; hat denotes an estimate; Pr: probability; $|$: conditional on; $\approx$: approximately.
- Read: among cases assigned about 80% confidence, roughly 80% should be correct. All quantities are unitless.
- Define the event; fit on reviewed calibration cases; assess on untouched representative cases and repeat after system changes.

Statistical evidence must match the test design

Comparison	Approach	Unit that must remain paired
Binary correctness, two systems	McNemar test; discordant cases	Same case under both systems.
Mean score difference	Paired bootstrap interval	Independent case or case family.
Many permutations per case	Summarize within case first	Do not count permutations as new cases.

A small p-value does not establish safe use

- Report effect size and an uncertainty interval.
- Set a practically acceptable sensitivity tolerance before looking at results.
- Failure to reject zero difference does not prove equivalence.
- Account for multiple tested prompts, metrics and model comparisons.

Class Exercise: specify a controlled evaluation

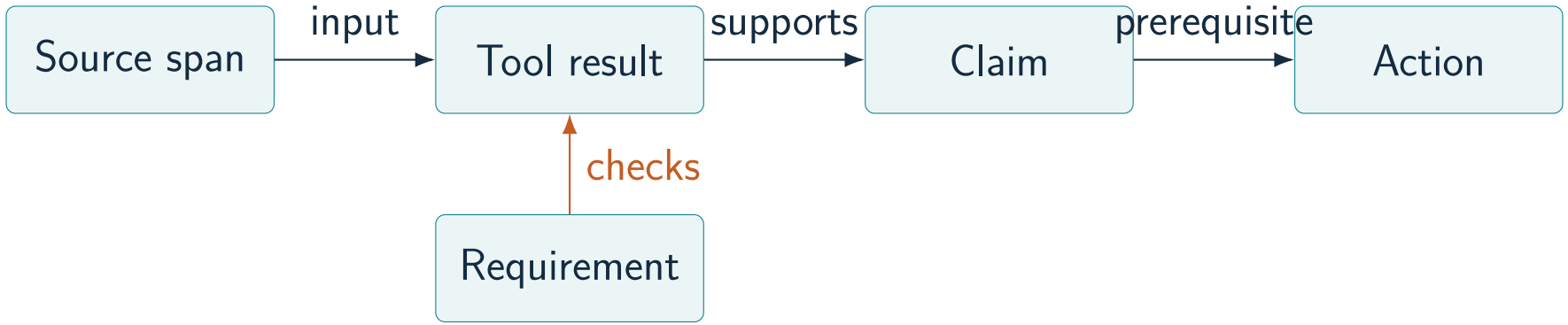
Step	Write down
1. Choose a claim	The answer should follow the supplied evidence.
2. Define three edits	Paraphrase; relevant contradiction; unrelated deletion.
3. Add a baseline	Repeat the identical prompt and exact evidence order.
4. Specify observations	Correctness, action flips and any available candidate scores.

Why inspect dependencies after scoring?

A failure rate tells us how often. A dependency trace helps identify what to repair.

Recorded object	Diagnostic question
Source span	Is this the fact the answer claims to use?
Tool input and output	Were the cited values actually passed to the calculator?
Derived claim	Which assumptions and prerequisites support it?
Proposed action	Which unresolved claims and permissions matter?

An evidence graph makes the repair path visible



- Record edge meanings and provenance; retain disputed links.
- A missing input marks downstream support unresolved. Recheck after repair.

Support, dependence and mechanism form a ladder

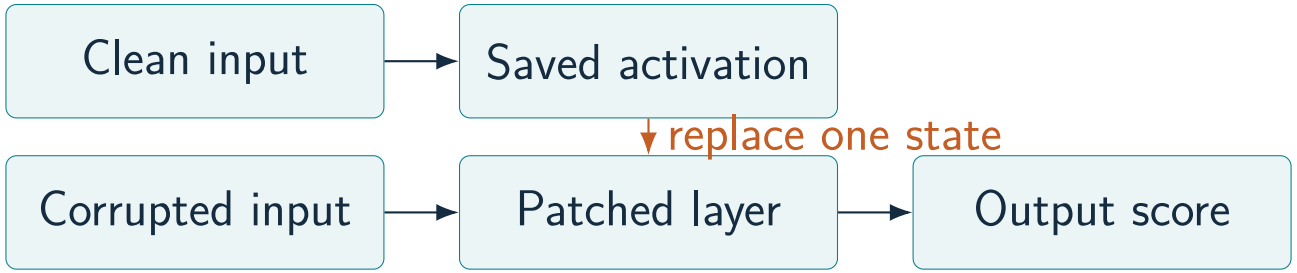
Claim	Evidence needed
The passage supports the answer	Reviewed entailment and eligibility.
Changing the passage changes the answer	Controlled input intervention.
A particular hidden state mediates the effect	Internal intervention with controls.
The repair improves deployment outcomes	Independent outcome evaluation.

Remove evidence; then retain only that evidence

$$\mathrm{Comp} = p_y(x) - p_y(x \setminus R), \qquad \mathrm{Suff} = p_y(x) - p_y(R).$$

- x: complete input; R: selected rationale evidence; $x \setminus R$: input with that evidence removed.
- y: fixed target answer; $p_y(z)$: its model probability under input z. Keep task instructions when retaining only R.
- Comp = comprehensiveness score drop on removal; Suff = sufficiency score drop using only the rationale. Differences are unitless probability points.
- Larger Comp suggests stronger removal effect; smaller Suff suggests retained support. Compare unrelated removals; redundancy and changed input distribution limit causal interpretation.

Patching asks a narrower internal question



$$\Delta = s(\text{corrupted run with patch}) - s(\text{corrupted run})$$

$s(\cdot)$ is a fixed output-score function; Δ is its patched-minus-original change. Use the same score and input; units follow the score. A positive change means the patch increased it.

Use the graph to select tests, then verify them

- Start from a failed claim and identify its source spans and tool inputs.
- Test a source-level repair before intervening inside the model.
- If needed, map the changed text to token positions for a controlled patch.
- Retest the repaired workflow on independent cases.

Question: no answer changes across reorderings

Which conclusion is warranted?

- A. The model is reliable in general.
- B. No selected-answer changes occurred in these tested cases.
- C. Its candidate probabilities must also be unchanged.

Answer: B—state the tested boundary

- Report the cases, permutations, repeats and output parser.
- Measure probability movement separately when complete scores are available.
- Check correctness: stable errors remain errors.
- More general reliability requires broader cases and an explicit tolerance.

Class Exercise: inspect saved framing responses

- Open `06_evaluation.ipynb`; run setup C01 and case C02.
- C03: compare neutral, bullish and skeptical responses and repeats.
- Change the case or model, rerun C02–C03; then C04 for all-case metrics.
- C05: remove actual earnings-per-share evidence from the checker.
- Inspect correctness separately from stability. C05 does not regenerate model answers.

Recorded pilot: low sensitivity can coexist with errors

Observed in the expanded framing pilot	Interpretation
1,200 assessments across two models	A controlled synthetic evaluation, not population prevalence.
Bullish framing did not increase unsupported buys	No evidence for that specific directional effect in this test.
Qwen mishandled some evidence gaps; Llama passed the benchmark	Correctness and framing sensitivity must be reported separately.

What BeliefLens adds to this evaluation loop

Activity	Concrete output
Organize evidence and executed steps	Traceable nodes, relations, source versions and tool bindings.
Apply declared requirements	Unresolved support, contradictions and affected claims.
Compare controlled runs	Case-level differences linked to evidence changes.
Repair and recheck	Recorded repair, review owner and remaining failures.

Finish with an evaluation card

Record	Decision it enables
Task, population and reference process	Know where the result applies.
Case families, controls and model revision	Reproduce and interpret the comparison.
Correctness, support, sensitivity and intervals	Assess behavior and uncertainty separately.
Known gaps, owner and retest trigger	Repair, limit or withhold deployment.

Five objectives: what can you now demonstrate?

Objective	Evidence from this session
Define success	Write the task contract and abstention condition.
Choose measures	Separate correctness, support, stability and calibration.
Control the comparison	Specify paired changes, repeats and uncertainty.
Diagnose responsibly	Distinguish graph dependencies from internal causal tests.
Decide what follows	Name a repair, retest and bounded release claim.

Reading route: lectures and primary methods

- Choi, Stanford CS224N Lecture 11 (2026) — benchmark design, metrics and evaluation protocols.
- Ribeiro et al., CheckList (ACL 2020) — minimum, invariance and directional tests.
- DeYoung et al., ERASER (ACL 2020) — rationale support and perturbation diagnostics.
- Dror et al., Statistical Significance in NLP (ACL 2018) — selecting paired significance tests.
- Guo et al., Calibration (ICML 2017); Meng et al., Causal Tracing / ROME (NeurIPS 2022) — confidence and internal interventions.

Section 6: objectives revisited

Objective	What you should now be able to explain
Evaluation	Separate retrieval, answer, behaviour and action metrics.
Attribution	Trace a claim to observable sources, tools and checks.
Controls	Compare changed prompts with identical-repeat variation.
Interpretation	Keep measured results separate from proposed methods.

STAC AI BOOTCAMP NYC



Deployment and governance clinic

Matthew Dixon, PhD, FRM
BeliefLens

21 October 2026 | 4:15–5:00

matthew.dixon@belieflens.org

The decision this session makes

Notebook: `07_governance.ipynb`

- A plausible answer is only a proposal.
- An operating system must decide what may happen next.
- Our deliverable: a release rule, an accountable owner and a reproducible decision record.

Why this matters: Turn evaluation findings into an enforceable operating process.

Roadmap: from purpose to permitted action

1. Define scope, harms and decision rights.
2. Separate evidence checks from authority controls.
3. Use a graph to route gaps and disputed claims.
4. Recheck at execution; monitor and requalify.
5. Apply the design to one workflow.

Why this matters: Each concept produces a part of the deployment plan.

Five learning objectives

- 7.1 Specify the workflow boundary and allowed actions.
- 7.2 Map evidence requirements to checks, experts and records.
- 7.3 Explain when a runtime gate can guarantee authority limits.
- 7.4 Choose monitoring and requalification triggers.
- 7.5 Design a bounded pilot and test its governance controls.

Why this matters: Assess design decisions rather than memorized terminology.

Start with a system boundary

- Name the user, task, affected people and intended benefit.
- List inputs, tools, external services and possible actions.
- Separate recommendation, approval and execution rights.
- State excluded uses, reversal options and an accountable owner.

Why this matters: A risk assessment requires a defined system and consequence.

A lifecycle, not a final approval meeting

Function	Practical question
Govern	Who sets policy and owns the consequences?
Map	What is the task, context and potential harm?
Measure	What evidence tests performance and limitations?
Manage	What changes, restrictions or response follow?

Why this matters: Use findings to revise scope, controls or deployment.

Policy and mechanism are different

- **Policy:** which behaviors are allowed?
- **Mechanism:** what actually prevents a disallowed behavior?
- **Assumption:** which routes and components must the mechanism control?

Why this matters: A prompt instruction alone does not establish an authority boundary.

Two questions at every release

Evidence and meaning

Is the conclusion supported?

Are sources applicable?

Are material gaps unresolved?

Authority and current state

May this actor take this action?

Is approval still valid?

Has the relevant state changed?

Why this matters: Passing one side does not imply passing the other.

Specify requirements before scoring evidence

- **R1 — Decision:** define purpose, population, metric and period.
- **R2 — Evidence:** require eligible sources and declared coverage.
- **R3 — Inference:** test whether comparisons support the claim.
- **R4 — Authority:** require current permission and accountable release.

Why this matters: Enough evidence means meeting requirements for a stated decision.

Document the population behind the source

- Who or what was sampled, and who is missing?
- How were records collected and selected?
- Which uses and comparisons are supported?
- Which collection details are unknown?

Why this matters: A large source can still be unsuitable for the question.

Which is better: Giants or Patriots?

GOVERNANCE CASE · SAME QUESTION-FIRST STRUCTURE AS MARMITE / VEGEMITE

NEW YORK GIANTS

American football



NEW ENGLAND PATRIOTS

American football



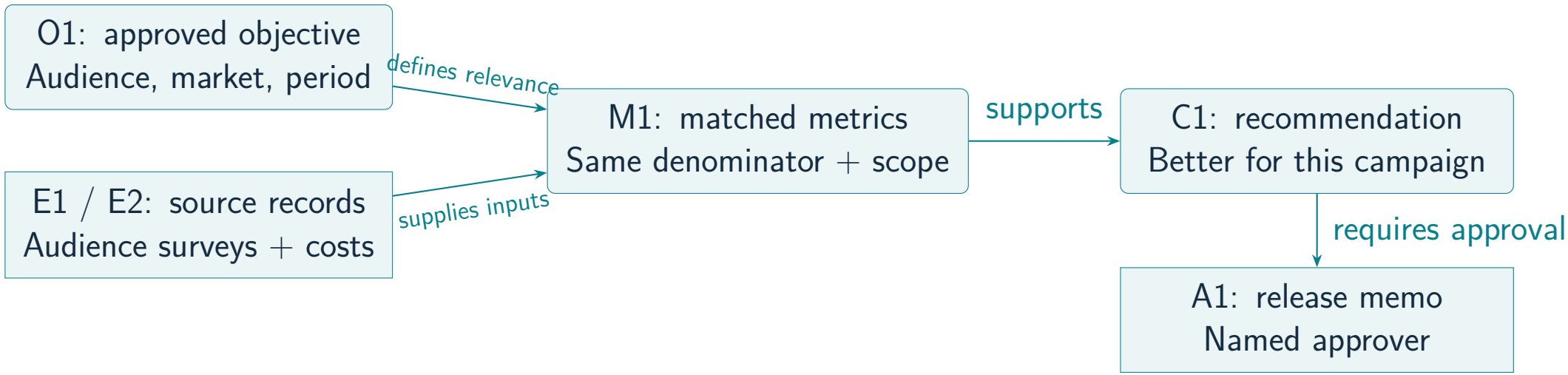
Human> Which team is better?

- Better for whom: a fan, sponsor or investor?
- Better on what: sporting success, audience reach or commercial return?
- Over which period, market and comparison population?

First decision: clarify the comparison before ranking the teams.

Map the question to its evidence and decision

HYPOTHETICAL SPONSORSHIP DECISION · NO TEAM RANKING IS ASSERTED



Rule attachments: R1 at O1; R2 at E1/E2; R3 on $E \rightarrow M \rightarrow C$; R4 at A1.

The governance matrix: checks, experts and decisions

General rule	Specific check / graph target	Action + expert	Required record
R1: Define the decision	O1: Is "better" an agreed campaign objective?	Clarify; business owner	Signed metric, scope and period
R2: Check the evidence	E1/E2: One team's fan poll only? Missing equivalent source for the other?	Hold comparison; data steward + survey specialist	Sampling frame, provenance, coverage gaps
R3: Validate the inference	$E \rightarrow M \rightarrow C$: Same population, units and period? Does reach imply return?	Qualify or escalate; statistician + commercial expert	Matched comparison, uncertainty and assumptions
R4: Authorize the action	A1: Material checks resolved and approval current?	Release or withhold; named approver	Rule version, check results and signed decision

People define the policy. Software evaluates declared checks, traces impact and routes exceptions.

So what decision does the graph support?

- **Clarify:** no agreed meaning of "better" ⇒ request objective and scope.
- **Escalate:** unmatched audiences or missing source methods ⇒ route the affected comparison to experts.
- **Withhold:** unresolved material support gap ⇒ block the team-ranking claim, not every unrelated fact.
- **Release:** comparable evidence, bounded claim and current approval ⇒ publish the scoped memo with limitations.

Illustrative governed response

"Define the campaign audience and success metric. Withhold the ranking until material coverage gaps are resolved."

Discussion: Trace one failed requirement to its owner and allowed action.

What the graph contributes

- A typed record: source, measurement, claim and proposed action.
- A dependency: which claim uses which source or calculation?
- A rule attachment: where is a requirement checked?
- A repair scope: which dependent checks must run again?

Why this matters: Localize the affected decision instead of rejecting every output.

Question: many sources, one population

- A comparison uses 100 surveys, all from one team's fan club.
- A. Release: the source count is large.
- B. Ask a model whether the evidence feels balanced.
- C. Check the target population; escalate the coverage gap.

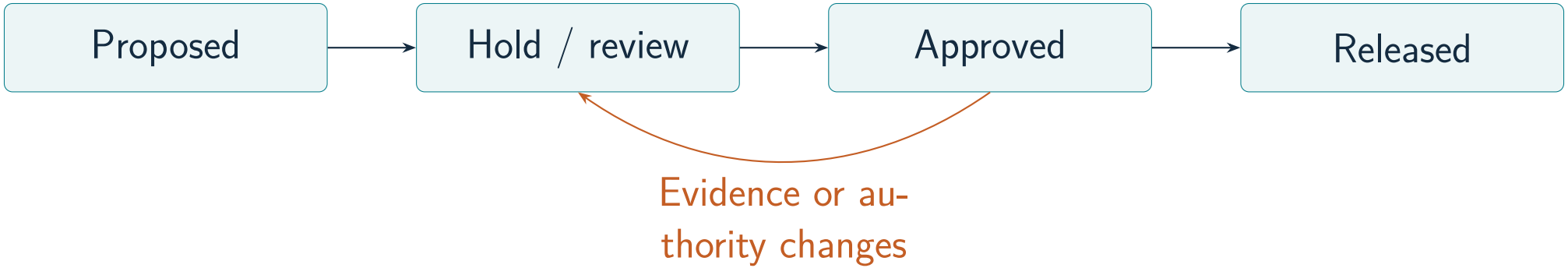
Why this matters: Apply R2 to the sampling frame, not the number of documents.

Answer: C — check coverage

- The question concerns a wider audience than the sample.
- Record the missing population coverage and affected claim.
- The survey specialist assesses a valid replacement or narrower claim.

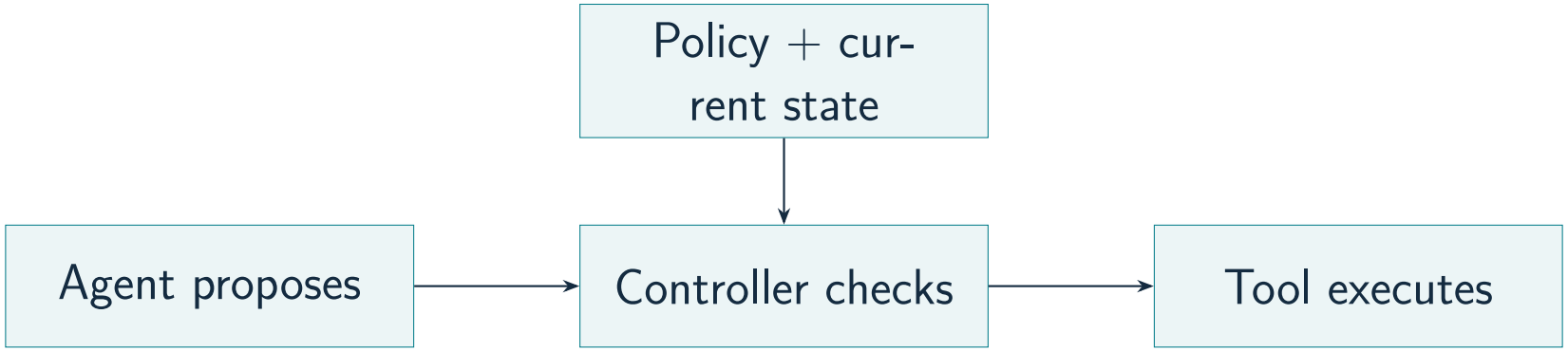
Why this matters: Unknown representativeness is a reason to qualify or withhold the comparison.

From a check to an operational state



Why this matters: Resolution and approval are separate transitions.

Place enforcement at the action boundary



Why this matters: Every protected action must pass through the controller.

A commit-time predicate

$$\text{allow}(a, s) = E(a, s) \wedge P(a, s) \wedge V(a, s) \wedge U(a, s).$$

- a: proposed action; s: current system state; allow: Boolean permission to execute; $\wedge$: logical AND (all tests must pass).
- E: required evidence checks passed; P: current permission and approval cover the action.
- V: source and state versions still match; U: action has not already been committed. Every test returns true or false, with no physical unit.
- Read: permit only if all prerequisites hold now. Requires protected state and an atomic check-and-dispatch boundary.

What could actually be proved?

- Specified invariant: no committed action exceeds its valid grant.
- Assumptions: every execution is mediated; checks and state are correct.
- Required controls: no bypass credentials; protected policy; atomic checks.
- Outside the claim: truth of evidence, business success, unknown threats.

Why this matters: A guarantee is only as strong as its model and assumptions.

Question: approved at 08:55

- At 08:59 the grant is revoked. Execution is scheduled for 09:00.
- A. The earlier approval is enough.
- B. Recheck current authority and block the action.
- C. Ask the language model to choose.

Why this matters: Distinguish an earlier recommendation from current authority.

Answer: B — recheck at execution

- A saved approval is evidence of a past decision.
- The controller checks the live grant and bound proposal.
- A revoked grant blocks the operation and records the reason.

Why this matters: Authorization must remain valid when the action occurs.

When do MDPs become relevant?

Structure	Appropriate use
Fixed predicate	A known prerequisite permits or blocks an action.
State machine	Earlier approvals, retries and commits affect current permission.
MDP	Choose sequential interventions using state, transitions and costs.
POMDP	Relevant state is partly hidden; observations update a belief.

Why this matters: Use sequential optimization when actions change future information or consequences.

A decision policy still needs hard limits

$$\pi(s) \in \mathcal{A}_{\text{allowed}}(s), \qquad V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{T} \gamma^{t} r_t \mid s_0 = s\right].$$

- s: state; π: action-selection policy; $\mathcal{A}_{\text{allowed}}(s)$: permitted actions there; $\in$: belongs to.
- $V^{\pi}(s)$: expected cumulative value under that policy; $\mathbb{E}_{\pi}$: average over resulting trajectories; s_0: initial state.
- t: step index; T: last step; r_t: reward in declared utility/cost units; $\gamma \in [0, 1]$: unitless discount; Σ: sum; $|$: conditional on.
- Read: choose within authority, then assess expected discounted consequences. Rewards and transitions need validation; an agent need not use reinforcement learning.

Question: which needs sequential reasoning?

- A. Block a tool call with an expired grant.
- B. Verify that a signed source exists.
- C. Decide whether another investigation is worth its delay and cost.

Why this matters: Recognize when the next action changes future options.

Answer: C — value of more information

- Investigation can change the evidence available for a later decision.
- Its value depends on uncertainty, cost and consequences.
- A and B can be enforced using fixed checks.

Why this matters: Add model complexity only when it improves the decision design.

Test the controller as well as the model

- Valid case: release should succeed under unchanged conditions.
- Fault cases: missing evidence, stale state, revoked grant, duplicate request.
- Measure false releases, false holds and review burden by failure type.
- Keep independent expected outcomes and a held-out test set.

Why this matters: An impressive model score does not establish a working release gate.

Monitor what invalidates qualification

- Input health: missing fields, source coverage and changed distributions.
- System health: timeouts, missing traces, stale models and service changes.
- Decision health: unsupported releases, override rates and delayed outcomes.
- Version health: model, prompt, corpus, tools, policy and grants.

Why this matters: Deployment approval does not survive every change automatically.

A trigger starts diagnosis, not automatic retraining

Finding	First response
Missing or stale source	Repair ingestion / retrieval; recheck affected claims.
Tool schema or calculation error	Fix integration; rerun relevant tests.
Task behavior fails with valid inputs	Review prompt or adaptation data; compare on held-out cases.
Scope or policy changes	Reassess permitted use and approval requirements.

Why this matters: Change the failing component and requalify the affected workflow.

Class Exercise: authorization must survive until execution

- Open `07_governance.ipynb`; run setup C01.
- C02: prepare and commit; verify one order and no duplicate dispatch.
- C03: prepare afresh, revoke the grant, commit; inspect the failed check.
- Change `grant` to `pending` or `snapshot`; rerun C03.
- C04: run all state changes; compare decisions, order counts and audit records.

Design your own governed pilot

APPLY 7.1, 7.2, 7.3, 7.4, 7.5

Work in pairs; report a decision and its evidence.

- Choose one workflow; state its user, inputs, output, prohibited action and accountable owner.
- Define two blocking rules, one escalation path, a safe-control case and two fault cases.
- Name the logs, change triggers and pilot exit criteria. Present a release decision under one injected fault.

What the pilot must produce

- A scope and authority record: objective 7.1.
- A requirements-to-graph-to-owner matrix: objective 7.2.
- A gate and valid/fault replay results: objective 7.3.
- Monitoring, change triggers and fallback: objective 7.4.
- A decision to release, restrict or continue testing: objective 7.5.

Why this matters: An operational plan is inspectable and testable.

Reading map: governance and enforcement

- Cornell CS513: policies, mechanisms and abstraction.
- Cornell CS5430: complete mediation and trusted components.
- Cornell CS513: runtime security automata.
- NIST AI RMF: lifecycle, scope and accountability.

Why this matters: Established systems concepts supply the foundations for agent governance.

Reading map: decisions, data and monitoring

- Berkeley CS188: states, actions, transitions and value.
- Datasheets for Datasets: collection, composition and uses.
- Google ML Crash Course: monitoring production pipelines.

Why this matters: Use the references to deepen a specific deployment decision.

Section 7: objectives revisited

Objective	What you should now be able to explain
Ownership	Assign evidence checks and exceptions to named roles.
Permission	Recheck current authority when an action is committed.
Monitoring	State which changes trigger review and revalidation.
Pilot	Define the permitted use, evidence records and stopping conditions.

Building trust: key takeaways

Trace the claim. Connect each material conclusion to eligible, timely evidence.

Locate the failure. Separate retrieval, interpretation, calculation and action selection.

Test the whole workflow. Use held-out cases, repeat controls and realistic failure conditions.

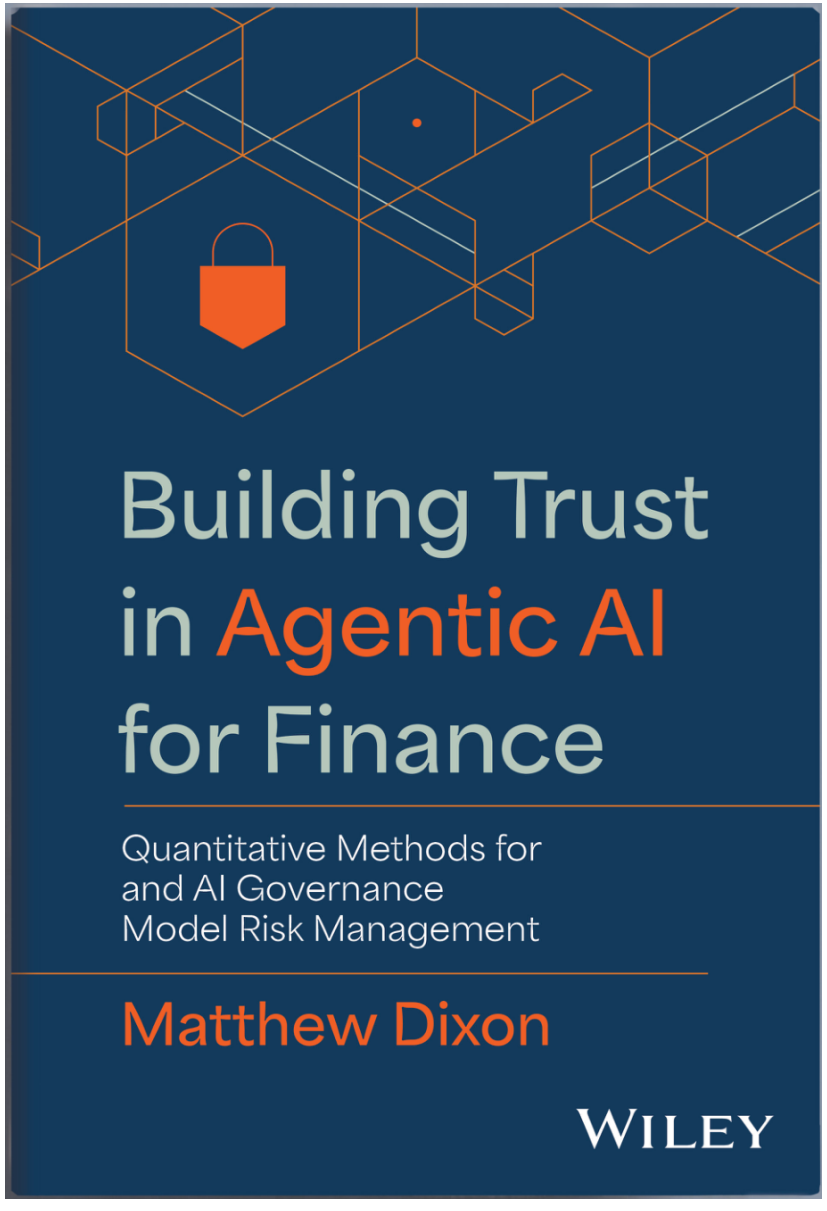
Measure uncertainty honestly. Token likelihood is not calibrated truth or expected profit.

Control the action. Enforce permissions, escalate gaps and record human accountability.

Your feedback

Please complete the supplied feedback form.

- Content, clarity and financial relevance.
- Your AI priorities and governance needs.
- Optional follow-up discussion.



FORTHCOMING FROM WILEY

Building Trust in Agentic AI for Finance

Quantitative Methods for AI Governance and Model Risk Management

Matthew Dixon, PhD, FRM